

JavaScript 知识详解（面试精华版）

🚀 **前端面试必备** — JavaScript 核心知识全面梳理，涵盖数据类型、闭包、原型链、异步编程、ES6+、Web API 等核心考点 | 建议收藏 ⭐

📌 知识脑图



📄 内容目录 (Table of Contents)

- 📦 [一、数据类型](#)
- 🔧 [二、ES6](#)
- 🏠 [三、JavaScript 基础](#)
- 🔗 [四、原型与原型链](#)
- 🎯 [五、执行上下文 / 作用域链 / 闭包](#)
- 🎯 [六、this / call / apply / bind](#)
- ⌚ [七、异步编程](#)
- 🏗️ [八、面向对象](#)
- 🗑️ [九、垃圾回收与内存泄漏](#)

- 📁 十、事件循环 (Event Loop) 详解
- 🌟 十一、现代JavaScript新特性
- 🌐 十二、浏览器 Web API
- 📄 总结

一、数据类型

1 JavaScript 有哪些数据类型，它们的区别？

JavaScript 共有 八种数据类型，分为 原始类型 (Primitive) 和 引用类型 (Reference) 。

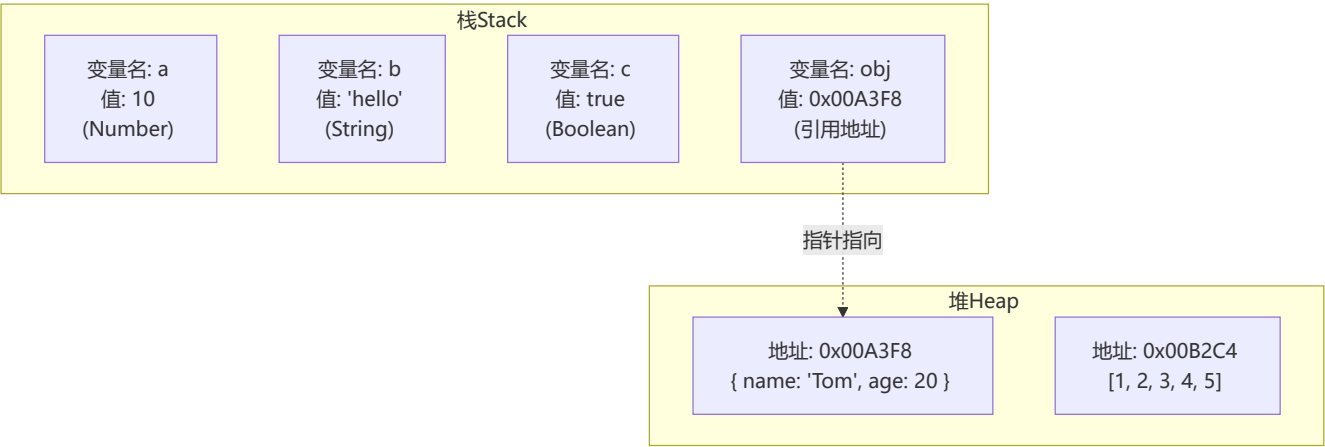
原始类型 (7种)：Undefined、Null、Boolean、Number、String、Symbol、BigInt

引用类型 (1种)：Object (包含普通对象、数组、函数、Date、RegExp、Map、Set 等)

其中 Symbol 和 BigInt 是 ES6 中新增的数据类型：

- **Symbol**：创建后独一无二且不可变的数据类型，主要解决全局变量冲突问题。每个 Symbol() 返回值都是唯一的，即使传入相同描述。
- **BigInt**：可表示任意精度格式的整数，使用 BigInt 可以安全地存储和操作大整数，超出 Number.MAX_SAFE_INTEGER (2^53 - 1) 范围也不会丢失精度。

栈 (Stack) vs 堆 (Heap) 存储

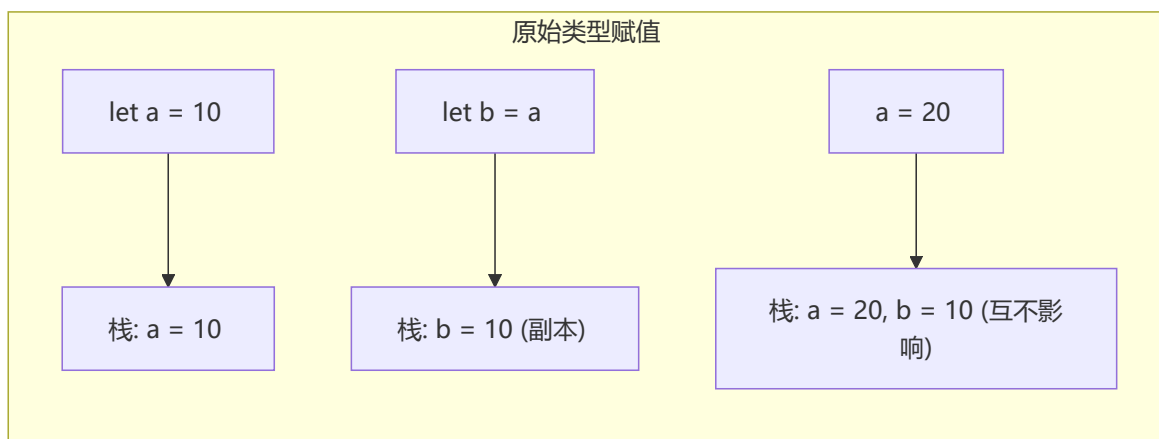
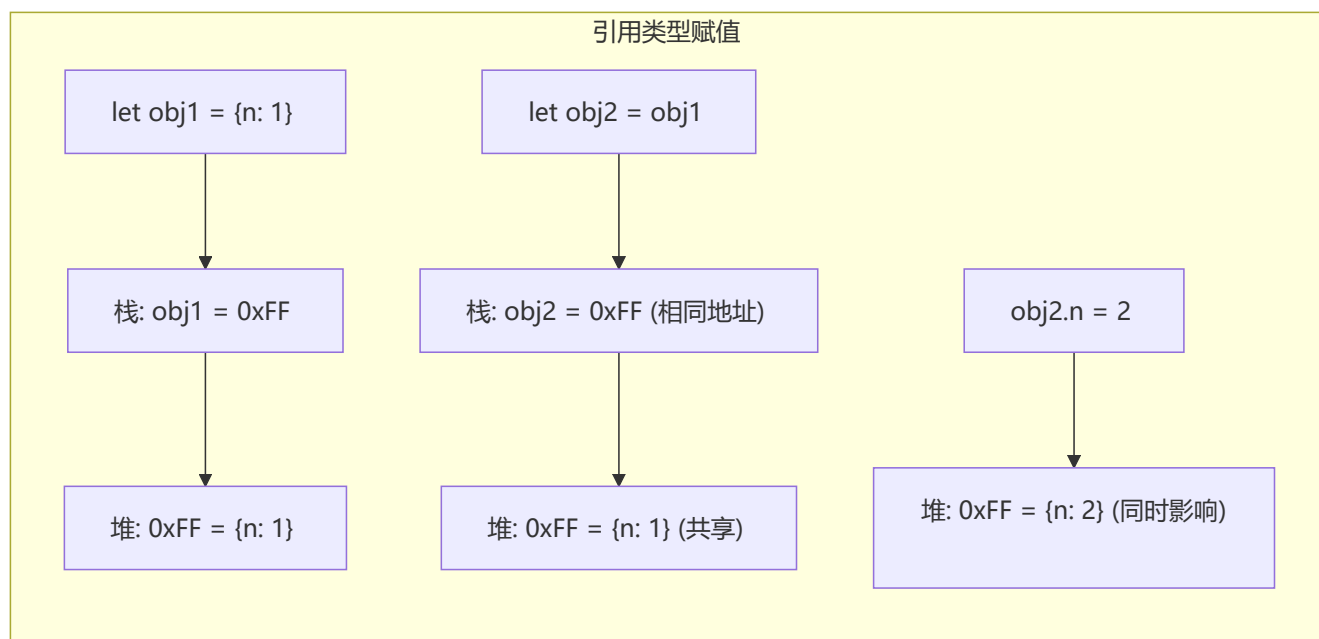


两种类型的区别在于存储位置的不同：

特性	原始类型	引用类型
存储位置	栈 (Stack)	堆 (Heap)
空间大小	固定、小	不固定、大
访问方式	按值访问	按引用访问
复制行为	独立副本	共享引用
比较方式	值比较	引用比较

详细说明:

- **原始数据类型** 直接存储在栈中的简单数据段，占据空间小、大小固定，属于被频繁使用数据，所以放入栈中存储。
- **引用数据类型** 存储在堆中的对象，占据空间大、大小不固定。如果存储在栈中，将会影响程序运行的性能。引用数据类型在栈中存储了**指针**，该指针指向堆中该实体的起始地址。当解释器寻找引用值时，会首先检索其在栈中的地址，取得地址后从堆中获得实体。



2 数据类型检测的方式有哪些

1 typeof

```
// typeof 操作符检测数据类型
console.log(typeof 2);           // number
console.log(typeof true);        // boolean
console.log(typeof 'str');       // string
console.log(typeof []);          // object
console.log(typeof function(){}); // function
console.log(typeof {});          // object
console.log(typeof undefined);   // undefined
```

```
console.log(typeof null);           // object (历史遗留bug)
console.log(typeof Symbol());       // symbol
console.log(typeof 10n);            // bigint
```

⚠ 注意: `typeof null` 返回 "object" 是 JavaScript 第一版就存在的 bug, 因为 `null` 的机器码全 0, 与 `object` 的类型标签相同。由于历史兼容性原因, 这个 bug 从未被修复。

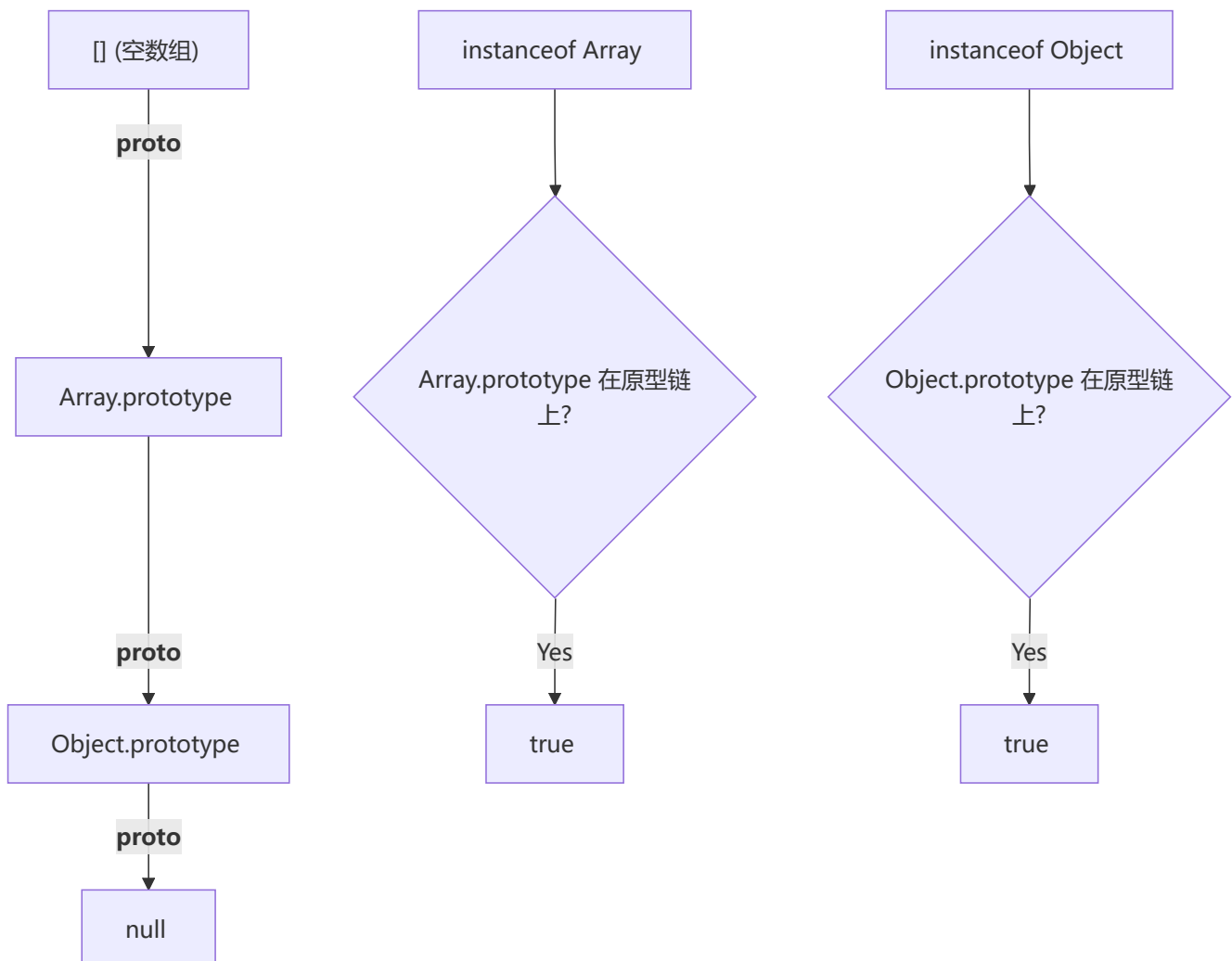
typeof 的局限性:

- `null` 被检测为 `object` (JS 第一版遗留bug, `null` 的机器码全0, 与 `object` 的类型标签相同)
- 数组、对象都被检测为 `object`, 无法区分具体对象类型

2 instanceof

```
// instanceof 检测构造函数原型是否在原型链上
console.log(2 instanceof Number);           // false
console.log(true instanceof Boolean);        // false
console.log('str' instanceof String);        // false
console.log([] instanceof Array);            // true
console.log(function(){} instanceof Function); // true
console.log({} instanceof Object);           // true
```

`instanceof` 的原理: 判断构造函数的 `prototype` 属性是否出现在对象的原型链中。



3 constructor

```
console.log((2).constructor === Number);           // true
console.log((true).constructor === Boolean);        // true
console.log(('str').constructor === String);        // true
console.log([]).constructor === Array;              // true
console.log((function() {}).constructor === Function); // true
console.log({}.constructor === Object);             // true
```

注意: `constructor` 可以被改写, 不安全。

```
function Fn(){};
Fn.prototype = new Array();
var f = new Fn();
console.log(f.constructor === Fn);    // false
console.log(f.constructor === Array); // true
```

4 Object.prototype.toString.call() (最准确)

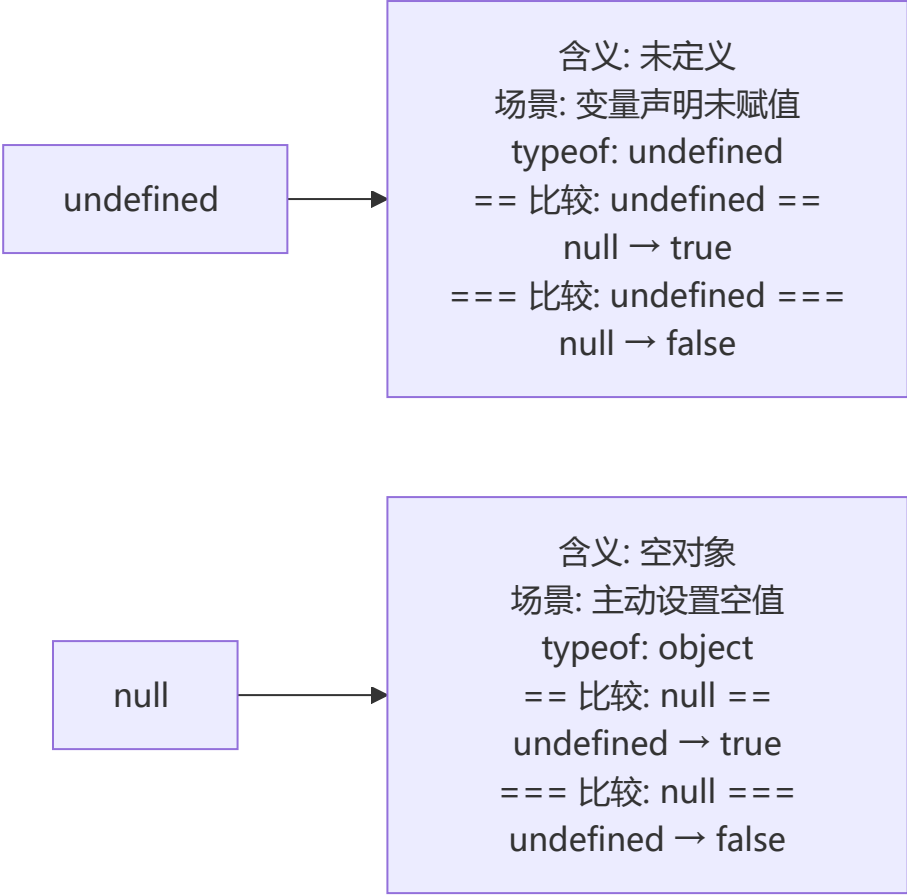
```
var a = Object.prototype.toString;
console.log(a.call(2));           // [object Number]
console.log(a.call(true));        // [object Boolean]
console.log(a.call('str'));       // [object String]
console.log(a.call([]));          // [object Array]
console.log(a.call(function(){})); // [object Function]
console.log(a.call({}));          // [object Object]
console.log(a.call(undefined));   // [object Undefined]
console.log(a.call(null));        // [object Null]
```

原理: 每个对象都有一个内部属性 `[[Class]]` (在ES5中), `Object.prototype.toString` 返回该属性的值。
`Array`、`Function` 等类型重写了 `toString` 方法, 所以必须通过 `Object.prototype.toString.call()` 来调用原始版本。

3 判断数组的方式有哪些

方法	代码示例	原理
<code>Object.prototype.toString.call()</code>	<code>Object.prototype.toString.call(obj).slice(8,-1) === 'Array'</code>	获取内部 <code>[[Class]]</code>
原型链	<code>obj.__proto__ === Array.prototype</code>	直接比较原型
<code>Array.isArray()</code>	<code>Array.isArray(obj)</code>	ES6 原生方法, 最推荐
<code>instanceof</code>	<code>obj instanceof Array</code>	检查原型链
<code>isPrototypeOf</code>	<code>Array.prototype.isPrototypeOf(obj)</code>	检查原型关系

4 null 和 undefined 区别



对比项	undefined	null
含义	未定义	空对象
类型	undefined	object (typeof)
转数字	NaN	0
JSON序列化	被忽略	保留为 null
常见场景	变量声明未赋值、函数无返回值	主动释放引用、原型链终点

`undefined` 在 JavaScript 中**不是保留字**，这意味着可以使用 `undefined` 作为变量名（极不推荐）。可以用 `void 0` 获取安全的 `undefined`。

5 typeof null 的结果为什么是 Object?

在 JavaScript 第一个版本中，所有值都存储在 32 位的单元中，每个单元包含一个小的**类型标签 (1-3 bits)**：

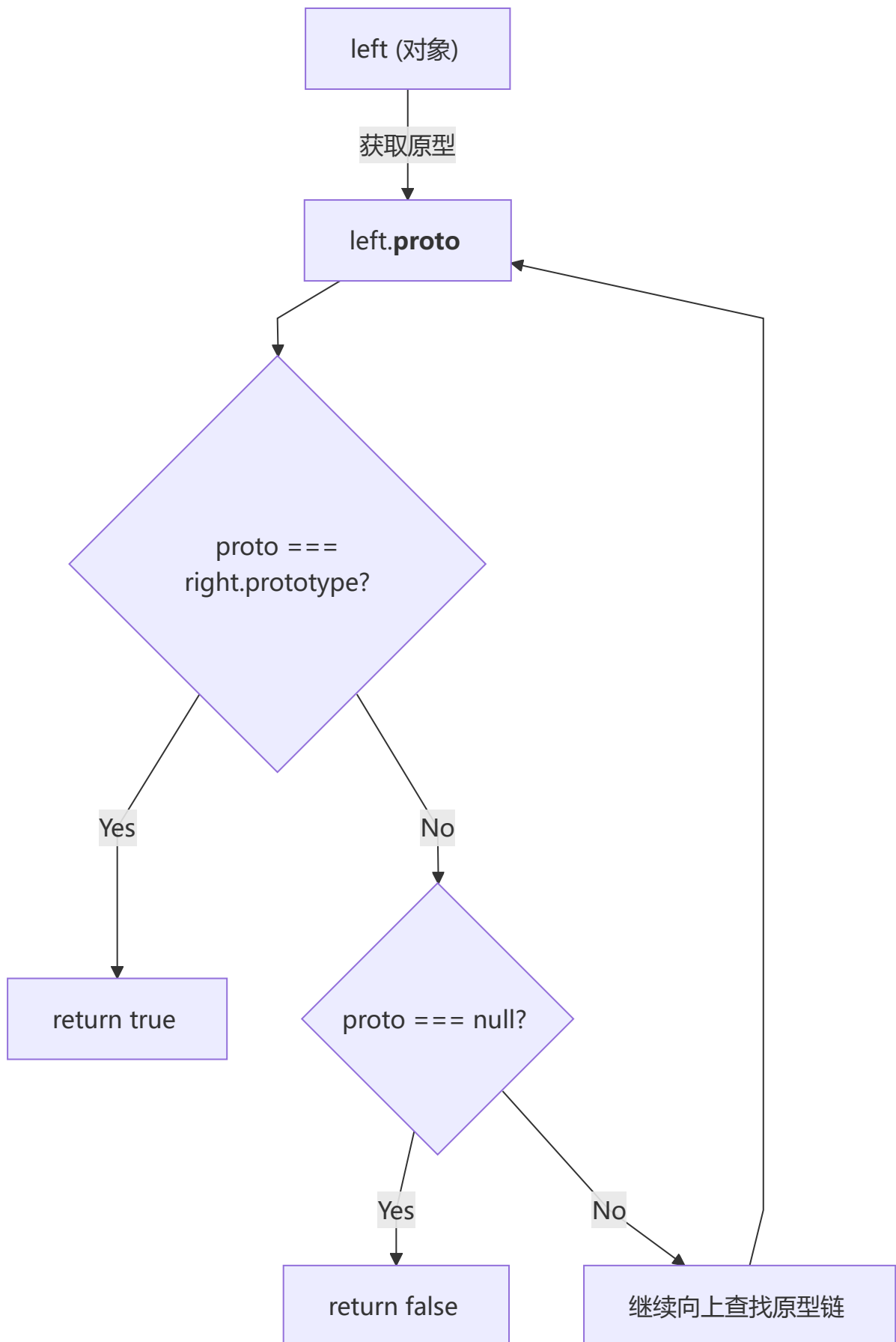
```
000: object
 1: int
010: double
100: string
110: boolean
```

`null` 的值是机器码 NULL 指针（全0），类型标签也是 `000`，和 `object` 类型标签一样，所以被判定为 `object`。这是 JS 设计之初的遗留 bug，但为了兼容性一直保留至今。

6 instanceof 操作符的实现原理

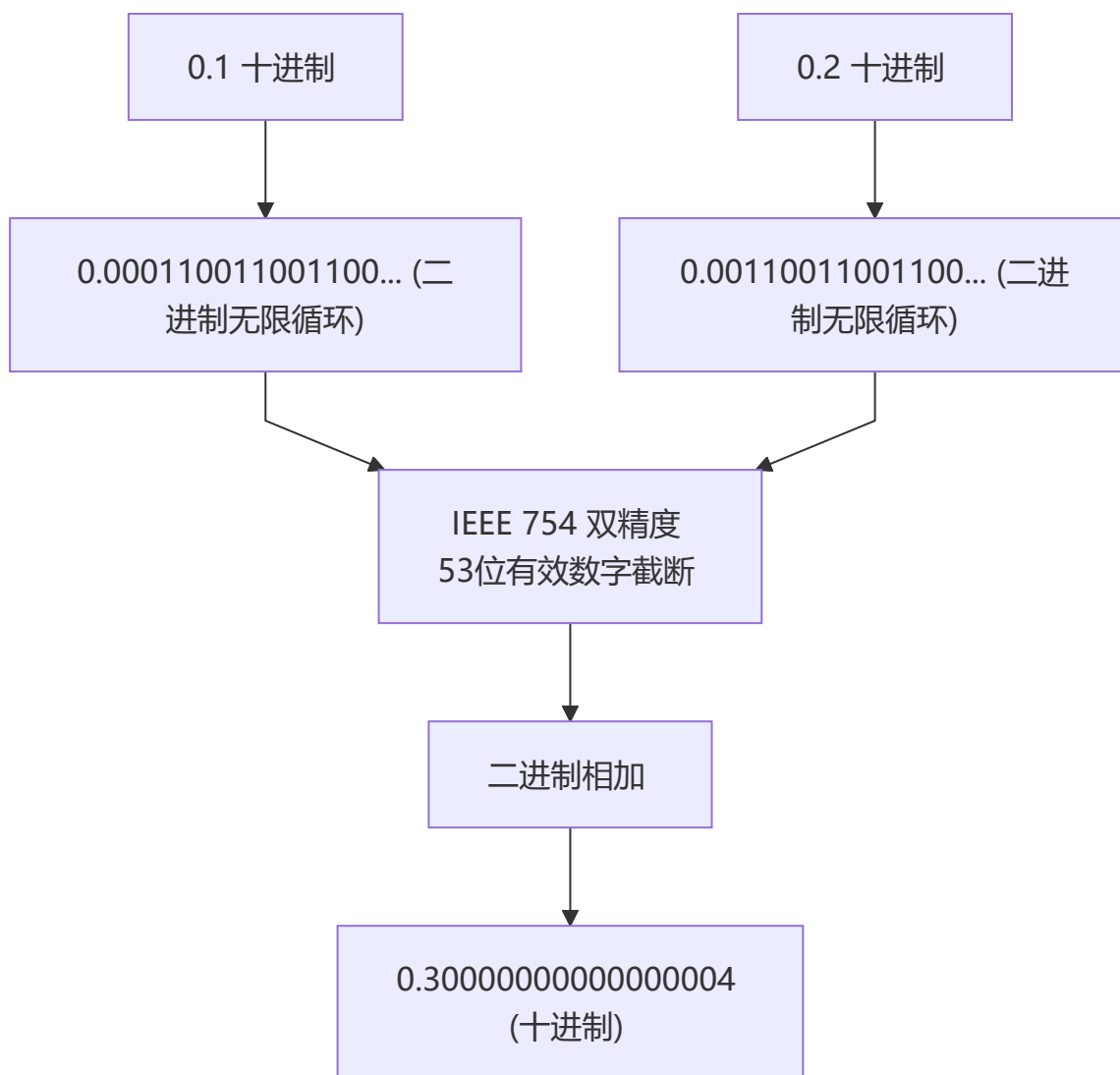
```
// myInstanceOf 手动实现 instanceof 原理
function myInstanceOf(left, right) {
  let proto = Object.getPrototypeOf(left)
  let prototype = right.prototype;

  while (true) {
    if (!proto) return false;
    if (proto === prototype) return true;
    proto = Object.getPrototypeOf(proto);
  }
}
```



7 为什么 $0.1 + 0.2 \neq 0.3$?

⚠ **经典面试题：**浮点数精度问题源于 IEEE 754 双精度标准。0.1 和 0.2 的二进制是无限循环小数，截断后相加产生误差。解决方案：`toFixed`、`Number.EPSILON` 或放大倍数计算。**注意：这不是 JS 特有的问题**，所有使用 IEEE 754 的语言都会遇到。



原因：JavaScript 的 Number 遵循 IEEE 754 标准，使用 64 位双精度浮点数表示。

- 符号位 (sign) : 1 bit
- 指数位 (exponent) : 11 bit
- 小数位 (fraction) : 52 bit

0.1 和 0.2 的二进制都是无限循环小数，IEEE 754 只能保留 53 位有效数字 (52 位 + 隐含的 1 位)，超出部分"0 舍 1 入"，导致精度丢失。

解决方案：

```
// 方法1: toFixed
(0.1 + 0.2).toFixed(2) // "0.30"

// 方法2: ES6 Number.EPSILON
function numberepsilon(arg1, arg2) {
  return Math.abs(arg1 - arg2) < Number.EPSILON;
}
console.log(numberepsilon(0.1 + 0.2, 0.3)); // true

// 方法3: 放大倍数后计算
(0.1 * 10 + 0.2 * 10) / 10 // 0.3
```

8 如何获取安全的 undefined 值？

```
void 0 // undefined
void (1 + 1) // undefined
void 'hello' // undefined
```

`void` 运算符对给定的表达式求值，然后返回 `undefined`。因为 `undefined` 可以被重写（非严格模式下），使用 `void 0` 最安全。

9 typeof NaN 的结果是什么？

```
typeof NaN // "number"
```

`NaN`（Not a Number）是一个“警戒值”，表示数学运算失败的结果。它是唯一一个与自身不相等的值：

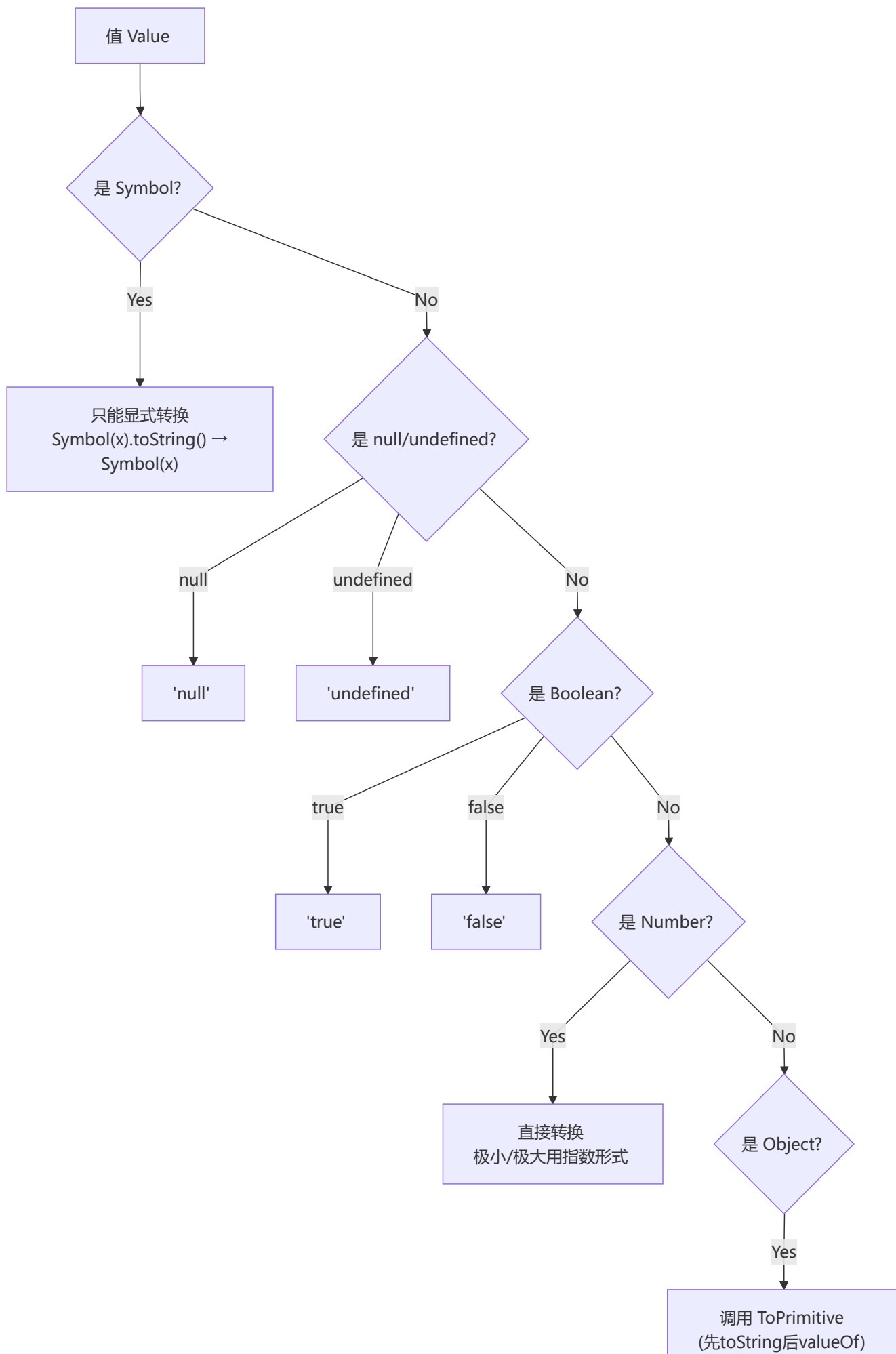
```
NaN === NaN // false
NaN !== NaN // true
```

10 isNaN 和 Number.isNaN 的区别

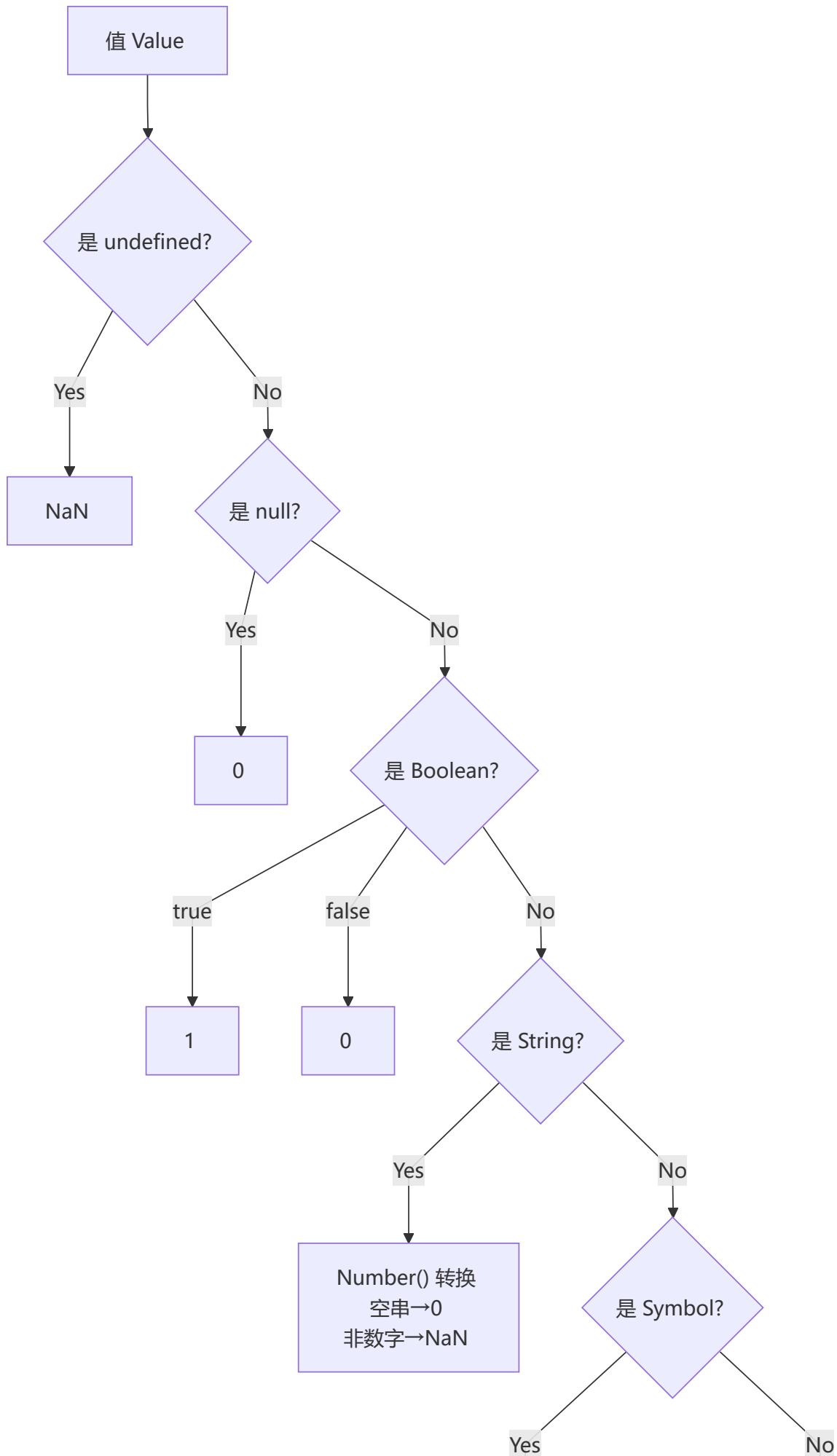
方法	行为	特点
<code>isNaN()</code>	先尝试转换为数字，再判断	非数字值也会返回 true，不准确
<code>Number.isNaN()</code>	先判断是否为数字类型，再判断	更准确，不进行类型转换

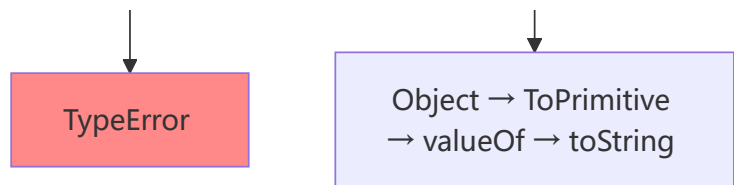
```
isNaN('hello') // true (先转换 Number('hello') → NaN, 然后判断)
Number.isNaN('hello') // false (先判断类型, 'hello' 不是 number, 直接 false)
Number.isNaN(NaN) // true
```

1 1 其他值到字符串的转换规则



1 2 其他值到数字值的转换规则





ToPrimitive 抽象操作流程:

当 `type = number` 时:

1. 调用 `valueOf()`，如果返回原始值，则返回
2. 否则调用 `toString()`，如果返回原始值，则返回
3. 否则抛出 `TypeError`

当 `type = string` 时 (仅 Date 对象默认):

1. 调用 `toString()`，如果返回原始值，则返回
2. 否则调用 `valueOf()`，如果返回原始值，则返回
3. 否则抛出 `TypeError`

1 3 其他值到布尔类型的转换规则

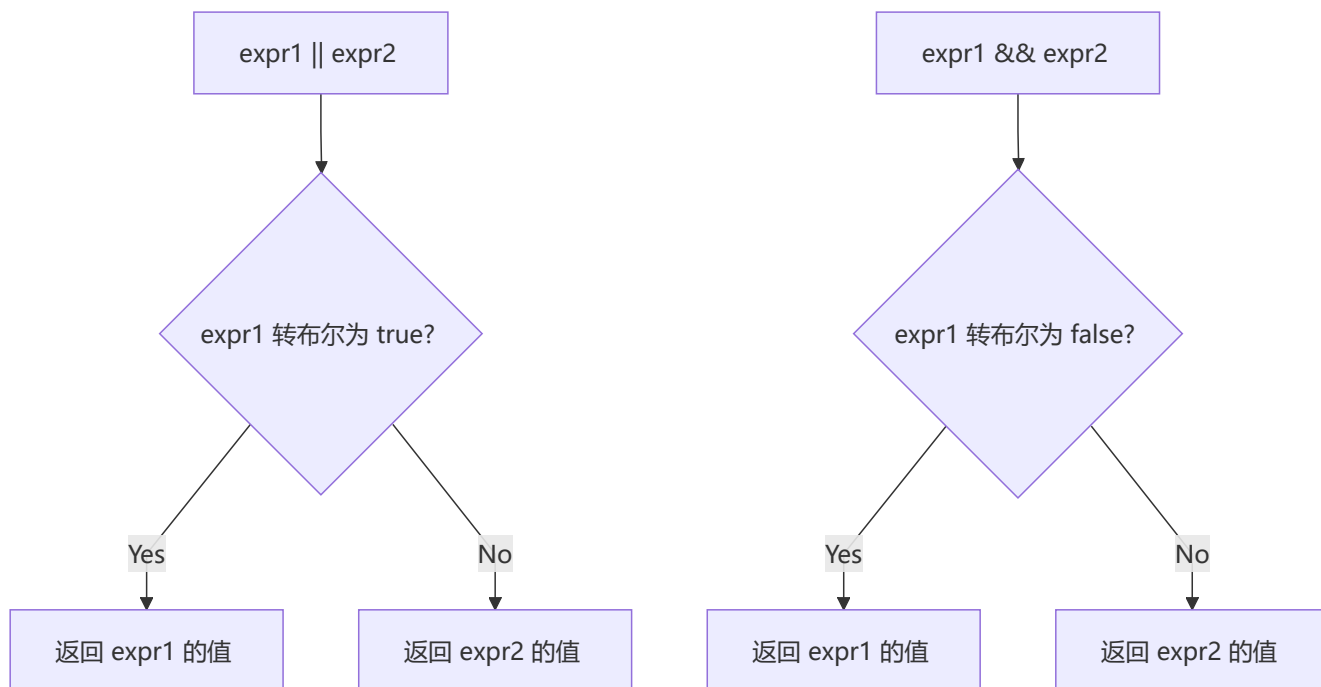
假值 (Falsy) 列表:

值	转布尔结果
<code>undefined</code>	<code>false</code>
<code>null</code>	<code>false</code>
<code>false</code>	<code>false</code>
<code>+0</code> 、 <code>-0</code> 、 <code>NaN</code>	<code>false</code>
<code>""</code> (空字符串)	<code>false</code>

所有其他值都是真值 (Truthy)，包括 `"false"` 字符串、`[]`、`{}`、`Infinity` 等。

1 4 || 和 && 操作符的返回值

这两个操作符返回的是操作数的值，而非布尔值。



```

0 || 'default'    // 'default'
'' || 'hello'     // 'hello'
'abc' && 123       // 123
null && 'anything' // null
  
```

1 5 Object.is() 与 == 和 === 的区别

比较方式	类型转换	特殊处理
<code>==</code>	会进行类型转换	<code>null == undefined</code> → true
<code>===</code>	不进行类型转换	严格相等
<code>Object.is()</code>	不进行类型转换	修复了 <code>===</code> 的两种特殊情况

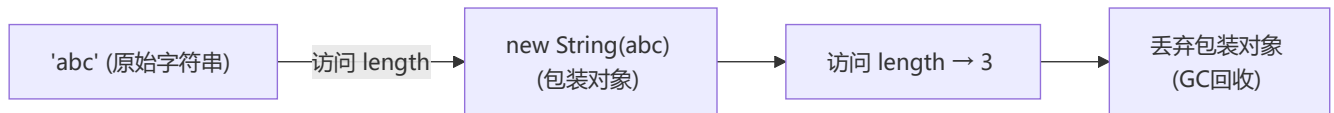
```

// 区别1: NaN 处理
NaN === NaN           // false
Object.is(NaN, NaN)   // true

// 区别2: +0 和 -0
+0 === -0             // true
Object.is(+0, -0)     // false
  
```

1 6 什么是 JavaScript 中的包装类型?

原始类型本身没有属性和方法，但 JavaScript 在访问原始类型的属性或方法时，会在后台隐式地将其转换为对应的包装对象。



```
const a = "abc";
a.length;           // 3 → 内部临时创建 String 包装对象
a.toUpperCase();    // "ABC"

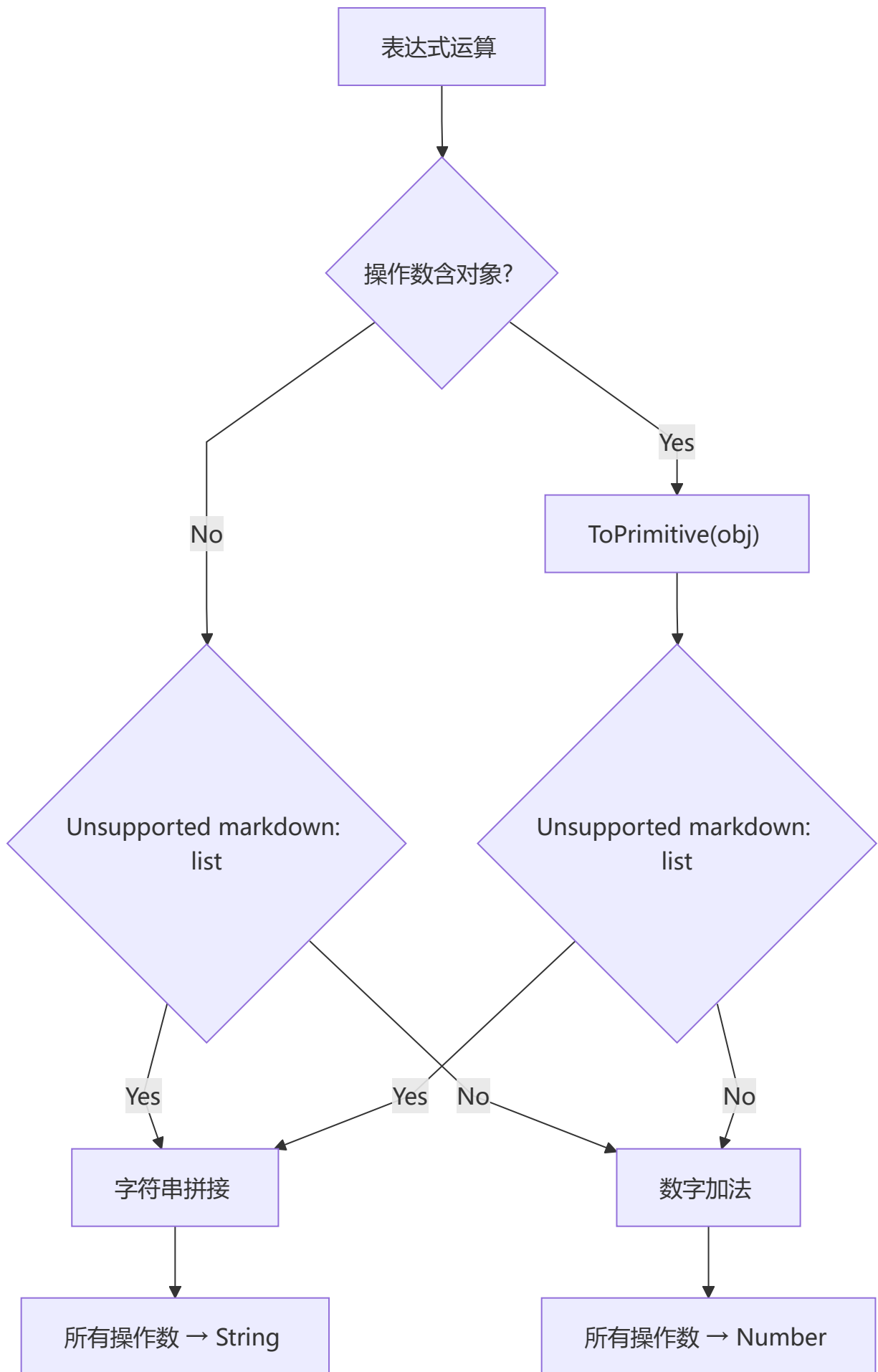
// 显式创建包装对象
Object('abc')       // String {"abc"}
new Number(123)     // Number {123}

// 包装对象转回原始值
var b = Object('abc');
b.valueOf()         // 'abc'
```

陷阱示例：

```
var a = new Boolean(false);
if (!a) {
  console.log("不会执行"); // a 是对象，对象永远是 truthy
}
```

1 7 JavaScript 中如何进行隐式类型转换?



+ 操作符规则:

- 有一方是字符串 → 字符串拼接
- 否则 → 数字加法

```
1 + '23'      // '123'
1 + false     // 1
'1' + false   // '1false'
false + true  // 1
```

-, *, / 操作符规则:

- 所有操作数转数字

```
1 * '23'      // 23
1 * false     // 0
1 / 'aa'      // NaN
```

== 操作符规则:

- 两边的值尽量转为 `number` 比较

```
3 == true      // false (3 → 3, true → 1)
'0' == false   // true  ('0' → 0, false → 0)
'0' == 0       // true  ('0' → 0)
```

1 8 + 操作符何时用于字符串拼接?

根据 ES5 规范, 如果某个操作数是字符串或能通过 `ToPrimitive` 转换为字符串, 则进行拼接。如果其中一个操作数是对象 (包括数组), 首先对其调用 `ToPrimitive(obj, number)`:

```
var a = {name: 'Jack'}
var b = {age: 18}
a + b // "[object Object][object Object]"

// 运算过程:
// a.valueOf() → {} (还是对象)
// a.toString() → "[object Object]"
// b.valueOf() → {}
// b.toString() → "[object Object]"
// 结果: "[object Object][object Object]"
```

1 9 为什么会有 BigInt 的提案?

`Number.MAX_SAFE_INTEGER = 9007199254740991` ($2^{53} - 1$)

超过这个范围, JavaScript 的数字计算会出现精度丢失:

```
9007199254740991 + 1 // 9007199254740992 (正确)
9007199254740991 + 2 // 9007199254740992 (错误, 应该是 9007199254740993)
```

BigInt 解决方案:

```
BigInt(9007199254740991) + 2n // 9007199254740993n
```

2 0 Object.assign 和扩展运算符: 深拷贝还是浅拷贝?

两者都是浅拷贝!

```
let outObj = {
  inObj: {a: 1, b: 2}
}

// 扩展运算符
let newObj1 = {...outObj}
newObj1.inObj.a = 2
console.log(outObj.inObj.a) // 2 (被修改了!)
```

```
// Object.assign
let newObj2 = Object.assign({}, outObj)
newObj2.inObj.a = 2
console.log(outObj.inObj.a) // 2 (也被修改了!)
```

区别:

- `Object.assign()`: 第一个参数为目标对象, 会触发 ES6 setter
- 扩展运算符 `...`: 不复制继承的属性和类的属性, 但复制 ES6 symbols 属性

2 1 如何判断一个对象是空对象?

```
// 方法1: JSON 序列化
JSON.stringify(obj) === '{}'
```

```
// 方法2: Object.keys (推荐)
Object.keys(obj).length === 0
```

```
// 方法3: 遍历 for...in + hasOwnProperty
function isEmpty(obj) {
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) return false
  }
  return true
}
```

```
// 方法4: Object.getOwnPropertyNames
Object.getOwnPropertyNames(obj).length === 0
```

二、ES6

1 let、const、var 的区别

💡 要点：`var` 有变量提升，可重复声明，无块级作用域；`let` / `const` 有块级作用域和暂时性死区（TDZ），不可重复声明。`const` 声明时必须初始化，且不能改变指针指向（但可修改对象属性）。

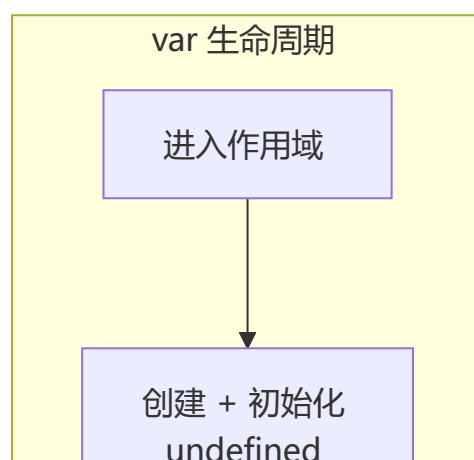
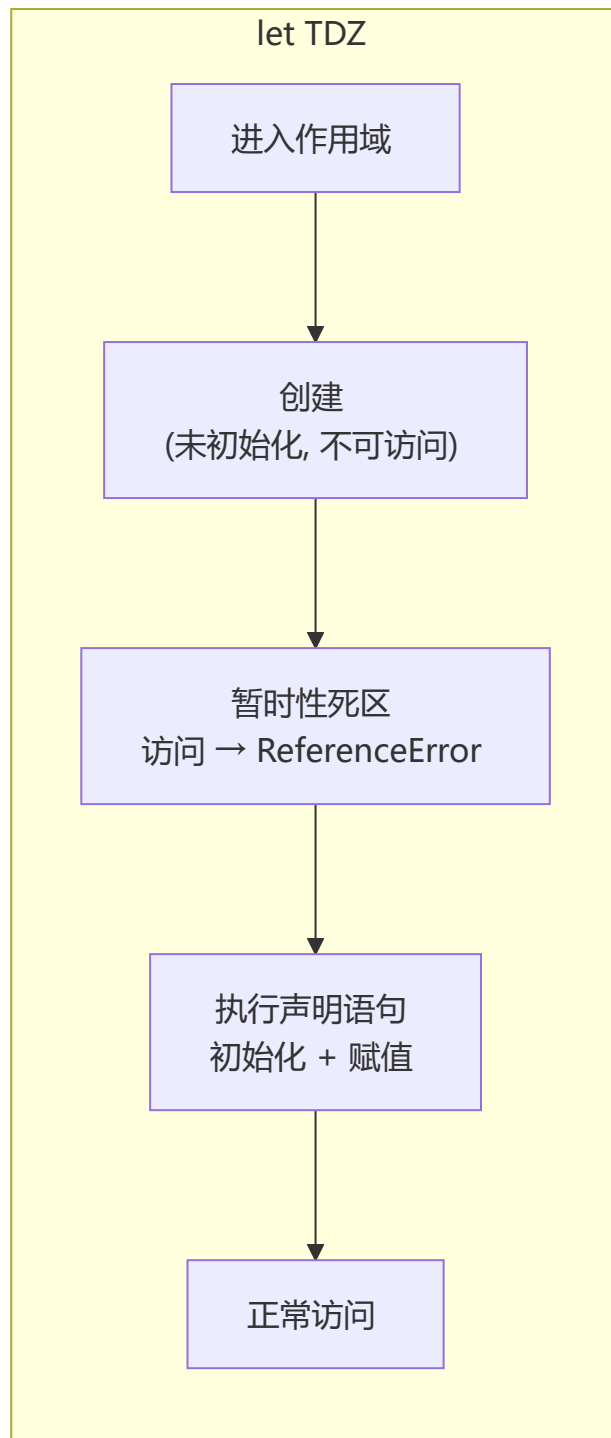
变量提升 块级作用域 可重复声明 挂载到 window 可重新赋值 必须设置初始化	变量提升 块级作用域 可重复声明 挂载到 window 可重新赋值	变量提升 块级作用域 可重复声明 挂载到 window 可重新赋值
---	---	---

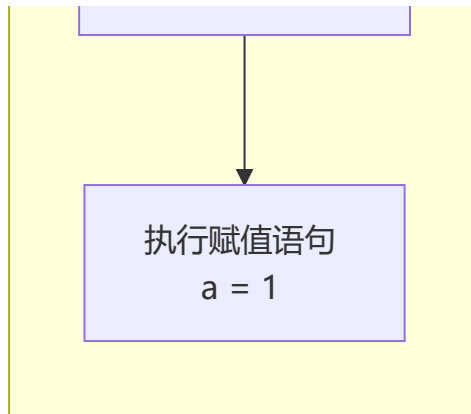
区别	var	let	const
块级作用域			
变量提升		(暂时性死区)	(暂时性死区)
挂载到全局对象	(window.varName)		
重复声明	(可覆盖)	(报错)	(报错)
暂时性死区			
必须设置初始值			
改变指针指向			

暂时性死区 (Temporal Dead Zone, TDZ) :

```
console.log(a) // undefined (var 提升)
var a = 1

console.log(b) // ReferenceError! (TDZ)
let b = 2
```





2 const 对象的属性可以修改吗？

可以修改属性，但不能重新赋值。

```
const obj = { name: 'Tom' }
obj.name = 'Jerry' // ✓ 属性修改允许
obj = {}           // ✗ TypeError: Assignment to constant variable

const arr = [1, 2, 3]
arr.push(4)        // ✓ 数组操作允许
arr = []           // ✗ TypeError
```

原理： `const` 保证的是变量指向的内存地址不能改变。对于原始类型，值就在栈内存中，所以值本身不可变。对于引用类型，栈中保存的是指针（堆地址），`const` 保证指针不变，但堆中的数据仍然可变。

3 如果 new 一个箭头函数会怎么样？

会报错： `arrowFn is not a constructor`

箭头函数不能作为构造函数，因为：

1. 箭头函数没有 `prototype` 属性
2. 箭头函数没有自己的 `this`，不能绑定到新对象
3. 箭头函数不能使用 `arguments`

4 箭头函数与普通函数的区别

箭头函数						普通函数				
没有自己的 <code>this</code> ✗ 从外层作用域继承	没有 <code>arguments</code> ✗ Unsupported markdown: code span 代理	没有 <code>prototype</code> ✗	不可 <code>new</code> ✗	不可用 <code>yield</code> ✗	<code>call</code> / <code>apply</code> / <code>bind</code> 无效 ✗	有自己的 <code>this</code> ✓	有 <code>arguments</code> ✓	有 <code>prototype</code> ✓	可作为构造函数 ✓	可用作 Generator ✓

(1) 简洁语法：

```
// 无参数
const fn1 = () => {}
// 单参数, 省略括号
const fn2 = x => x * 2
// 多参数
const fn3 = (a, b) => a + b
// 返回值是对象需要括号
const fn4 = () => ({ name: 'Tom' })
// 无返回值
const fn5 = () => void console.log('test')
```

(2) 没有自己的 this:

```
const obj = {
  id: 'OBJ',
  normal: function() { console.log(this.id) }, // this → obj
  arrow: () => { console.log(this.id) }        // this → 外层 (window/global)
}
obj.normal() // 'OBJ'
obj.arrow()  // undefined (严格模式) 或 全局 id
```

(3) this 不可改变:

```
const arrowFn = () => console.log(this)
arrowFn.call({id: 1}) // 还是指向外层 this
arrowFn.apply({id: 1}) // 还是指向外层 this
arrowFn.bind({id: 1})() // 还是指向外层 this
```

(4) 没有 arguments:

```
const normalFn = function() { console.log(arguments) }
normalFn(1, 2, 3) // Arguments(3) [1, 2, 3]

const arrowFn = (...args) => console.log(args) // 用 rest 参数替代
arrowFn(1, 2, 3) // [1, 2, 3]
```

5 箭头函数的 this 指向哪里?

箭头函数捕获其所在上下文的 `this` 值作为自己的 `this`, 且不可改变。

Babel 转译理解:

```
// ES6
const obj = {
  getArrow() {
    return () => console.log(this === obj)
  }
}

// ES5 (Babel 转译后)
```

```
var obj = {
  getArrow: function getArrow() {
    var _this = this // → 捕获外层 this
    return function() {
      console.log(_this === obj)
    }
  }
}
```

6 扩展运算符的作用及使用场景

对象扩展运算符

```
// 复制对象（浅拷贝）
let bar = { a: 1, b: 2 }
let baz = { ...bar } // { a: 1, b: 2 }

// 等价于 Object.assign
let baz2 = Object.assign({}, bar)

// 覆盖属性
let merged = { ...bar, ...{a: 2, b: 4} } // { a: 2, b: 4 }

// 修改对象的部分属性（Redux reducer 常用）
const newState = { ...state, user: { ...state.user, name: 'newName' } }
```

数组扩展运算符

```
// 参数序列
function add(x, y) { return x + y }
const numbers = [1, 2]
add(...numbers) // 3

// 复制数组（浅拷贝）
const arr1 = [1, 2]
const arr2 = [...arr1]

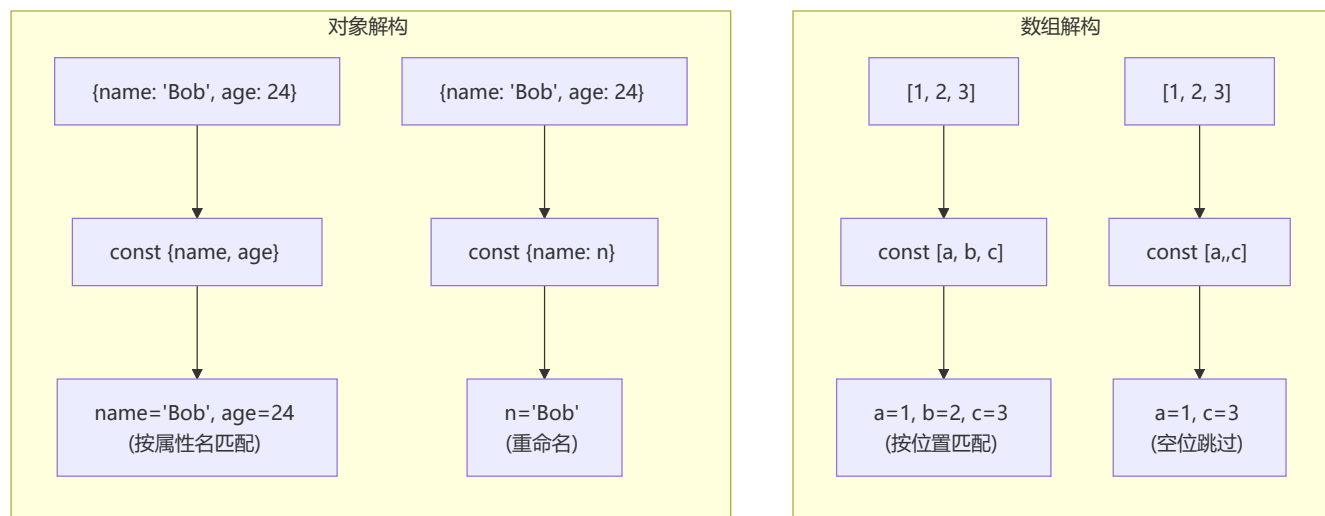
// 合并数组
const merged = ['one', ...arr1, 'four'] // ['one', 1, 2, 'four']

// 解构赋值
const [first, ...rest] = [1, 2, 3, 4] // first=1, rest=[2,3,4]

// 字符串转数组
[...'hello'] // ['h', 'e', 'l', 'l', 'o']

// 类数组转数组
function foo() {
  const args = [...arguments]
}
```

7 对对象与数组的解构的理解



8 提取高度嵌套的对象里的指定属性

```
const school = {
  classes: {
    stu: {
      name: 'Bob',
      age: 24
    }
  }
}

// 方法1: 逐层解构
const { classes } = school
const { stu } = classes
const { name } = stu

// 方法2: 一行解构 (推荐)
const { classes: { stu: { name } } } = school
console.log(name) // 'Bob'

// 解构 + 重命名
const { classes: { stu: { name: studentName } } } = school
console.log(studentName) // 'Bob'
```

9 对 rest 参数的理解

Rest 参数将多个独立参数收集到一个数组中:

```
// 收集剩余参数
function sum(...numbers) {
  return numbers.reduce((acc, cur) => acc + cur, 0)
}
sum(1, 2, 3, 4) // 10

// 与解构结合
const [first, second, ...rest] = [1, 2, 3, 4, 5]
// first=1, second=2, rest=[3,4,5]

// 注意: rest 参数必须在最后
// const [...rest, last] = [1,2,3] ❌ 语法错误
```

1 0 ES6 中模板语法与字符串处理

模板字符串特性:

1. `${}` 嵌入变量/表达式
2. 保留空格、缩进、换行
3. 支持运算表达式

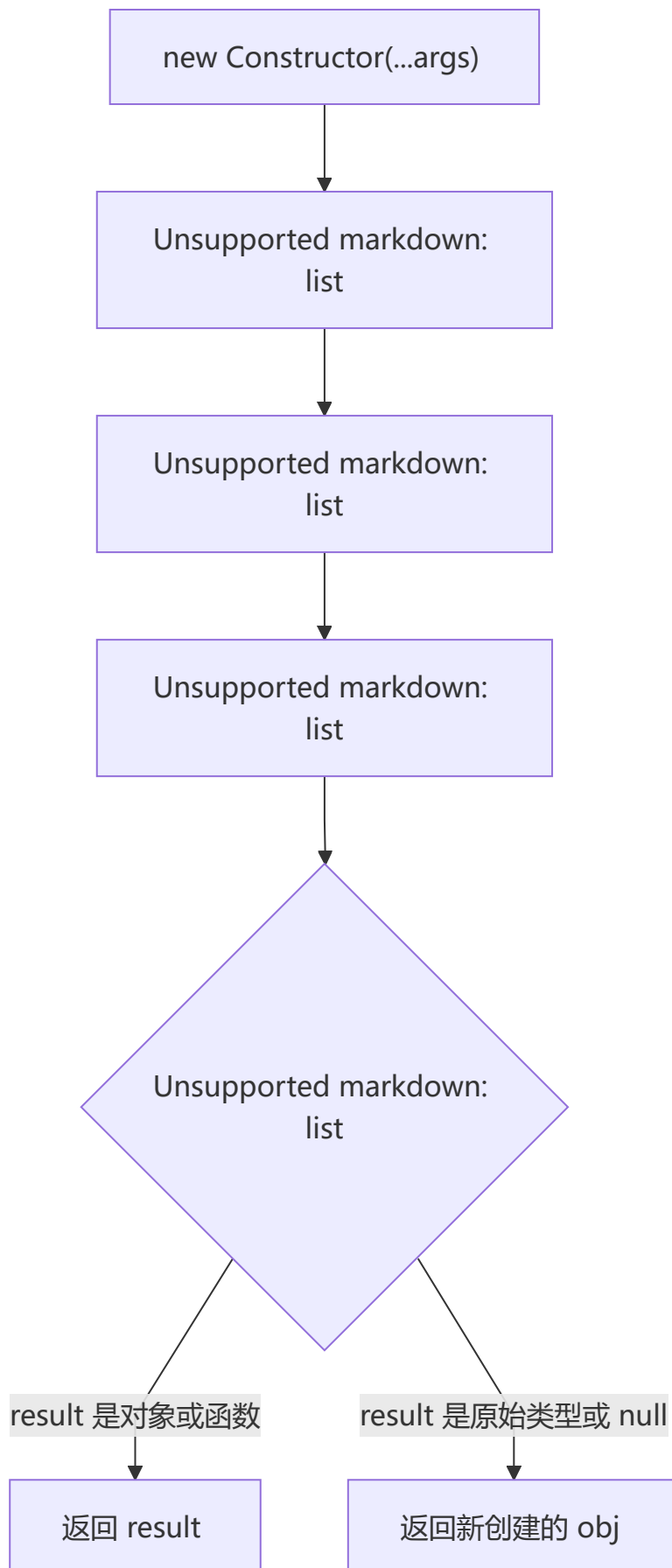
```
const name = 'css'
const career = 'coder'
const html = `
  <div>
    <h1>${name}</h1>
    <p>${career}</p>
    <p>${1 + 2}</p>
  </div>
`
```

新增字符串方法:

方法	作用	示例
<code>includes()</code>	是否包含子串	<code>'hello'.includes('ell')</code> → true
<code>startsWith()</code>	是否以某串开头	<code>'hello'.startsWith('he')</code> → true
<code>endsWith()</code>	是否以某串结尾	<code>'hello'.endsWith('lo')</code> → true
<code>repeat()</code>	重复字符串	<code>'ab'.repeat(3)</code> → 'ababab'

三、JavaScript 基础

new 操作符的实现原理



手动实现:

```
function objectFactory() {
```

```

let newObject = null
let constructor = Array.prototype.shift.call(arguments)
let result = null

if (typeof constructor !== "function") {
  console.error("type error")
  return
}

// 1+2: 创建空对象并链接原型
newObject = Object.create(constructor.prototype)

// 3: 绑定 this 并执行
result = constructor.apply(newObject, arguments)

// 4: 判断返回类型
let flag = result && (typeof result === "object" || typeof result === "function")
return flag ? result : newObject
}

```

2 Map 和 Object 的区别

特性	Map	Object
键的类型	任意类型（包括对象、函数）	String 或 Symbol
键的顺序	按插入顺序有序	无序（ES6 后部分有序）
size	<code>map.size</code> 直接获取	需手动计算 <code>Object.keys().length</code>
迭代	原生可迭代（ <code>for...of</code> ）	需先获取 keys
性能	频繁增删改查更优	未针对此场景优化
原型继承	无默认键	有原型链，可能产生冲突

3 Map 和 WeakMap 的区别

WeakMap					Map				
键: 必须是对象	弱引用 不影响 GC	不可迭代 ✖	无 size ✖	无 clear ✖	键: 任意类型	强引用	可迭代	有 size 属性	有 clear() 方法

WeakMap 弱引用机制：

```

let obj = { name: 'Tom' }
const wm = new WeakMap()
wm.set(obj, 'some data')

obj = null // 引用被删除
// WeakMap 中的键名对象会被 GC 自动回收，无需手动删除

```

应用场景：

- 存储 DOM 元素的私有数据
- 缓存数据（对象不再被使用时自动释放）
- 防止内存泄漏

4 JavaScript 有哪些内置对象

分类总结：

分类	示例
值属性	<code>Infinity</code> 、 <code>NaN</code> 、 <code>undefined</code> 、 <code>null</code>
函数属性	<code>eval()</code> 、 <code>parseInt()</code> 、 <code>parseFloat()</code> 、 <code>isNaN()</code>
基本对象	<code>Object</code> 、 <code>Function</code> 、 <code>Boolean</code> 、 <code>Symbol</code> 、 <code>Error</code>
数字日期	<code>Number</code> 、 <code>Math</code> 、 <code>Date</code>
字符串	<code>String</code> 、 <code>RegExp</code>
可索引集合	<code>Array</code> 、 <code>Int8Array</code> 、 <code>Uint8Array</code> 等
键值集合	<code>Map</code> 、 <code>Set</code> 、 <code>WeakMap</code> 、 <code>WeakSet</code>
结构化数据	<code>JSON</code> 、 <code>ArrayBuffer</code> 、 <code>DataView</code>
控制抽象	<code>Promise</code> 、 <code>Generator</code> 、 <code>Iterator</code>
反射	<code>Reflect</code> 、 <code>Proxy</code>
国际化	<code>Intl</code> 、 <code>Intl.Collator</code>

5 常用的正则表达式

```
// 16进制颜色
/#[0-9a-fA-F]{6}|[0-9a-fA-F]{3})/g

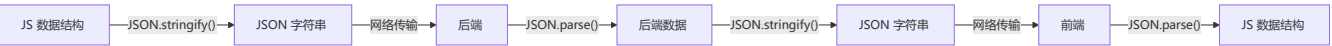
// 日期 yyyy-mm-dd
/^[0-9]{4}-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])$/

// QQ号
/^[1-9][0-9]{4,10}$/g

// 手机号
/^1[34578]\d{9}$/g

// 用户名（字母开头，4-16位字母数字下划线）
/^[a-zA-Z\$][a-zA-Z0-9_\$]{4,16}$/
```

6 对 JSON 的理解



JSON 与 JS 对象的区别:

- JSON 属性值不能为函数
- JSON 属性名必须用双引号
- JSON 中不能出现 `undefined`、`NaN`
- JSON 中不能有注释

JSON.stringify 的特殊处理:

```
JSON.stringify({a: undefined, b: function() {}, c: Symbol()})
// '{}' // 忽略 undefined、函数、Symbol

JSON.stringify({a: /abc/})
// '{"a":{}}' // RegExp 转空对象

JSON.stringify({a: NaN, b: Infinity, c: -Infinity})
// '{"a":null,"b":null,"c":null}' // 特殊数字转 null
```

7 JavaScript 脚本延迟加载的方式

方式	说明
<code>defer</code> 属性	与文档解析同步加载，文档解析完后按顺序执行
<code>async</code> 属性	异步加载，加载完成后立即执行（顺序不可预测）
动态创建 DOM	<code>document.createElement('script')</code> ，文档加载完后插入
<code>setTimeout</code>	设置定时器延迟加载
放在底部	<code></body></code> 前放置 script 标签

```
<!-- defer: 解析完 HTML 后执行，多个 defer 按顺序 -->
<script defer src="script.js"></script>

<!-- async: 加载完立即执行，不保证顺序 -->
<script async src="script.js"></script>
```

async 脚本

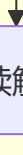
HTML 解析



遇到 script
后台下载



下载完毕
立即执行



继续解析

defer 脚本

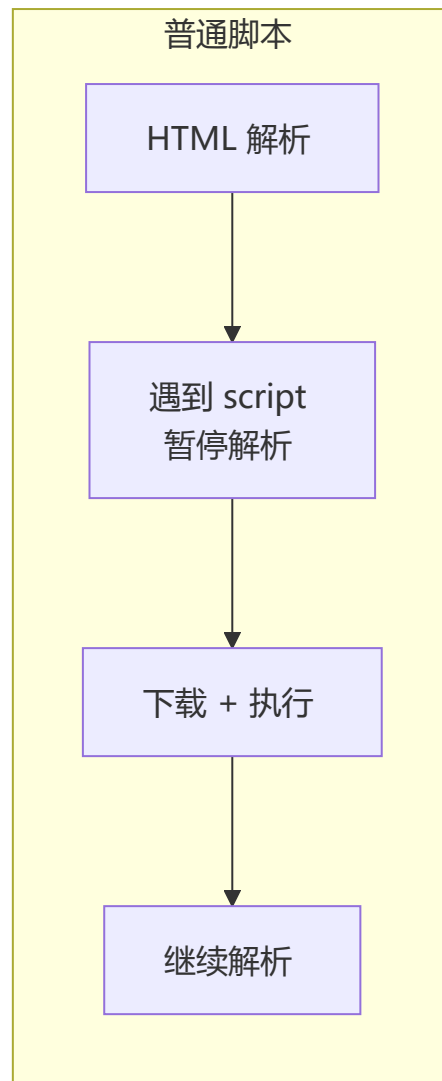
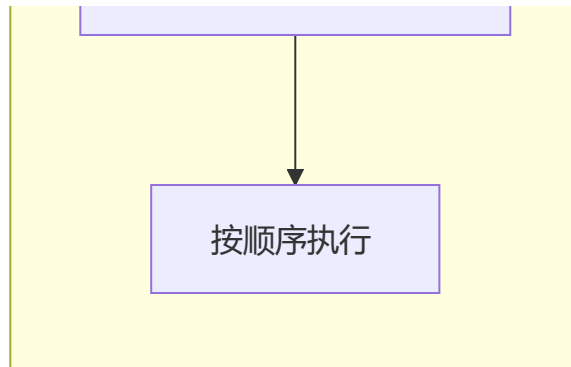
HTML 解析



遇到 script
后台下载



下载完毕
等待 HTML 解析完成



8 类数组对象的定义与转换

类数组对象： 拥有 `length` 属性和数字索引的对象。

```
// 常见类数组
arguments          // 函数的参数对象
document.querySelectorAll('div') // NodeList
document.forms      // HTMLCollection
```

转换为数组的四种方法：

```
// 1. Array.prototype.slice.call
Array.prototype.slice.call(arrayLike)

// 2. Array.prototype.splice.call
Array.prototype.splice.call(arrayLike, 0)

// 3. Array.prototype.concat.apply
Array.prototype.concat.apply([], arrayLike)

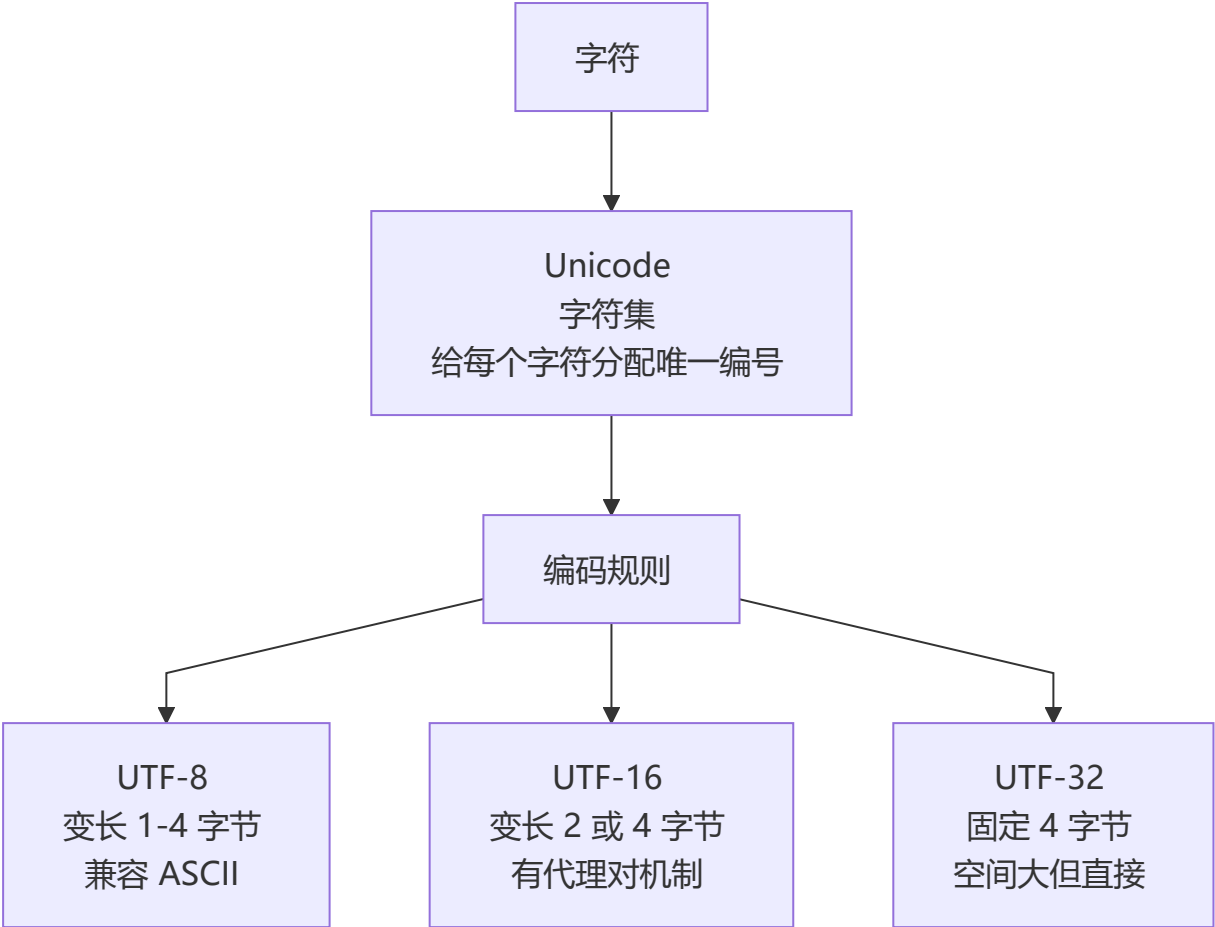
// 4. Array.from (ES6 推荐)
Array.from(arrayLike)

// 5. 扩展运算符 (需可迭代)
;[...arrayLike]
```

9 数组有哪些原生方法？

类别	方法	影响原数组	描述
转换	<code>toString()</code> 、 <code>join()</code>	✗	数组转字符串
尾部操作	<code>push()</code> 、 <code>pop()</code>	✓	尾部增删
首部操作	<code>shift()</code> 、 <code>unshift()</code>	✓	首部增删
排序	<code>reverse()</code> 、 <code>sort()</code>	✓	排序/反转
连接	<code>concat()</code>	✗	合并数组
截取	<code>slice()</code>	✗	返回子数组
增删改	<code>splice()</code>	✓	全能操作
查找	<code>indexOf()</code> 、 <code>lastIndexOf()</code> 、 <code>includes()</code>	✗	查找元素
迭代	<code>forEach()</code> 、 <code>map()</code> 、 <code>filter()</code> 、 <code>some()</code> 、 <code>every()</code>	✗	遍历处理
归并	<code>reduce()</code> 、 <code>reduceRight()</code>	✗	累计计算
扁平	<code>flat()</code> 、 <code>flatMap()</code>	✗	数组扁平化
填充	<code>fill()</code> 、 <code>copyWithin()</code>	✓	批量填充

1 0 Unicode、UTF-8、UTF-16、UTF-32 的区别



特性	Unicode	UTF-8	UTF-16	UTF-32
类型	字符集	编码规则	编码规则	编码规则
字节长度	-	1-4 可变	2 或 4	固定 4
兼容 ASCII	-	✓	✗	✗
中文占用	-	3 字节	2 字节	4 字节
容错性	-	差 (错一个影响后续)	好 (只错一个字符)	好

Unicode 平面概念:

- 基本平面 (BMP) : U+0000 ~ U+FFFF, 65536 个码位
- 辅助平面 (SMP) : U+10000 ~ U+10FFFF, 16 个平面

UTF-16 代理对机制:

```
// "嫡" 字 U+21800, 超出 BMP
// 计算: 0x21800 - 0x10000 = 0x11800
// 二进制: 0001000110 0000000000
// 高位 (U+D800 + 前10位) → 0xD846
// 低位 (U+DC00 + 后10位) → 0xDC00
// UTF-16 编码: 0xD846 0xDC00
```

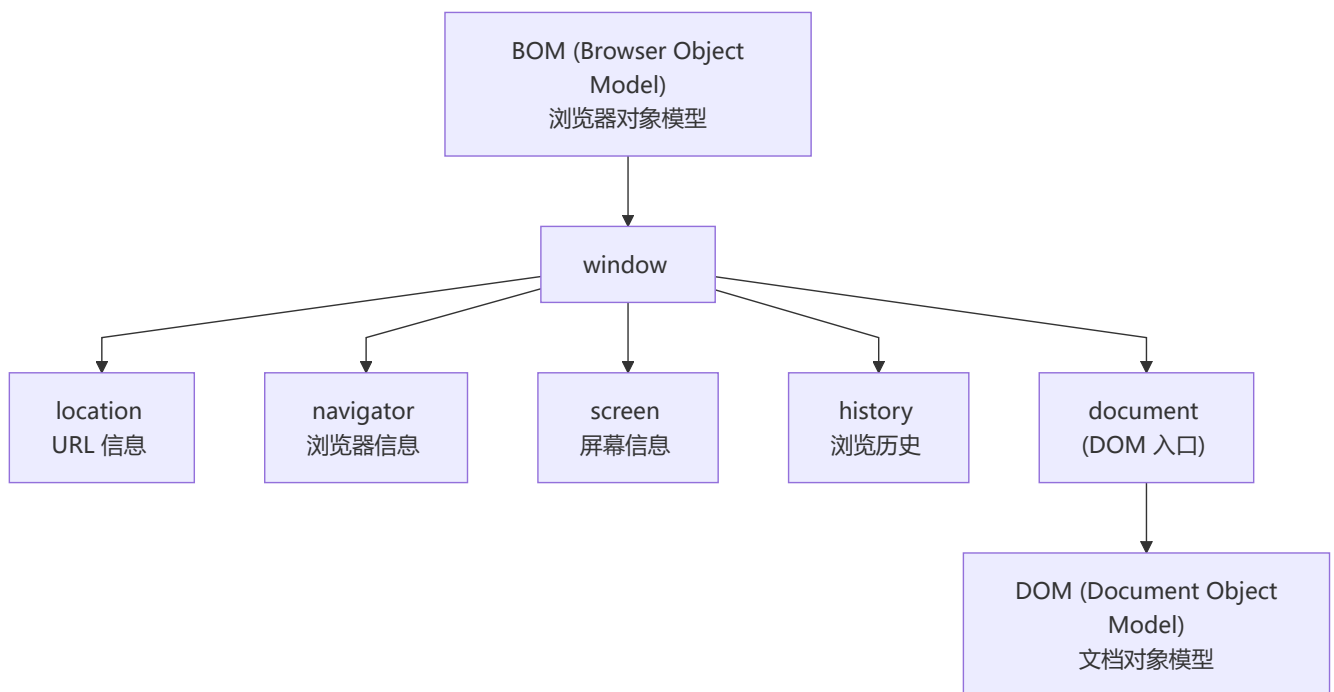
1 1 位运算符

运算符	名称	规则
&	按位与	两个位都为1 → 1
	按位或	两个位都为0 → 0
^	按位异或	相同为0, 相异为1
~	按位取反	0变1, 1变0
<<	左移	高位丢弃, 低位补0, 相当于乘 2^n
>>	右移	正数左补0, 负数左补1, 相当于除 2^n

1 2 为什么 arguments 是类数组而不是数组?

`arguments` 是一个对象, 有 `length` 和数字索引属性, 但**没有继承** `Array.prototype`, 所以不能调用 `push`、`forEach` 等数组方法。它是为了性能考虑而设计的特殊对象。

1 3 什么是 DOM 和 BOM?



DOM： 文档对象模型，定义了访问和操作 HTML 文档的 API（`getElementById`、`createElement`、`appendChild` 等）。

BOM： 浏览器对象模型，提供了与浏览器交互的 API（`alert`、`setTimeout`、`location`、`navigator` 等）。

1 4 escape、encodeURIComponent 的区别

```
const url = "https://example.com/测试 page?name=张三&age=20"

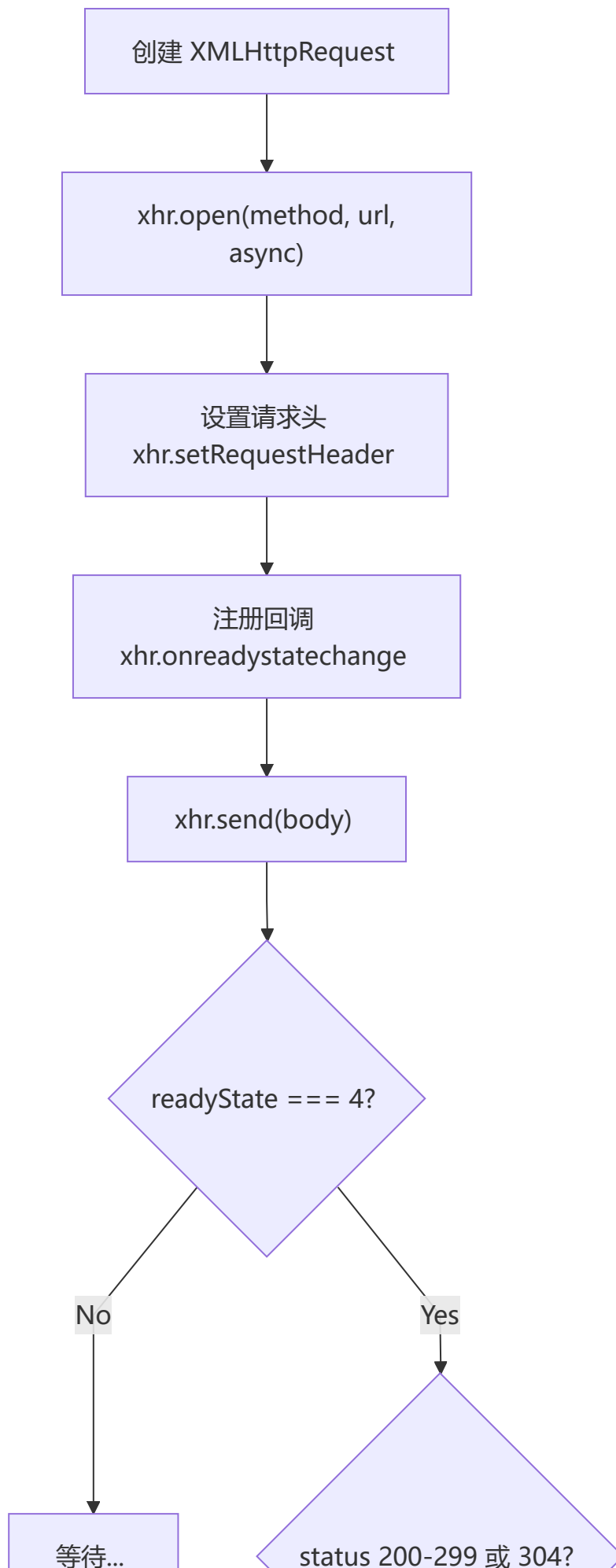
encodeURIComponent(url)
// "https://example.com/%E6%B5%8B%E8%AF%95%20page?name=%E5%BC%A0%E4%B8%89&age=20"
// 保留 URI 结构字符 (:/?#[ ]@!$&'()*+,-.~)

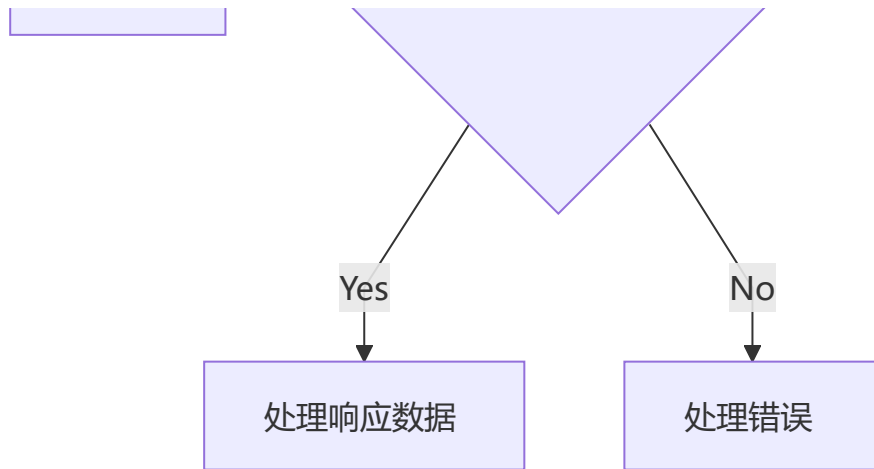
encodeURIComponent("测试 page?name=张三")
// "%E6%B5%8B%E8%AF%95%20page%3Fname%3D%E5%BC%A0%E4%B8%89"
// 编码所有非字母数字字符，包括 ?=& 等
```

函数	用途	编码范围
<code>encodeURIComponent</code>	编码整个 URL	不编码 URI 保留字符
<code>encodeURIComponent</code>	编码 URL 参数值	编码所有特殊字符
<code>escape</code> （已弃用）	编码字符串	对非 ASCII 字符编码为 %uxxxx

1 5 对 AJAX 的理解

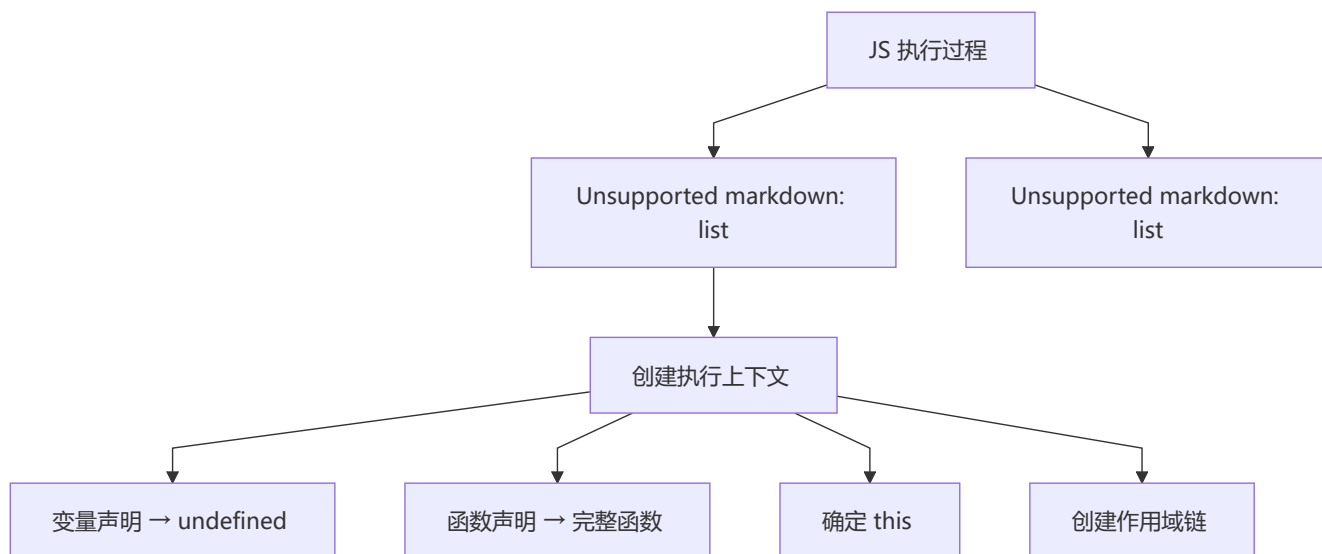
AJAX（Asynchronous JavaScript And XML）： 在不重新加载整个页面的情况下，与服务器交换数据并更新部分网页。





```
// Promise 封装 AJAX
function getJSON(url) {
  return new Promise(function(resolve, reject) {
    let xhr = new XMLHttpRequest()
    xhr.open("GET", url, true)
    xhr.onreadystatechange = function() {
      if (this.readyState !== 4) return
      if (this.status === 200) {
        resolve(this.response)
      } else {
        reject(new Error(this.statusText))
      }
    }
    xhr.onerror = function() {
      reject(new Error(this.statusText))
    }
    xhr.responseType = "json"
    xhr.setRequestHeader("Accept", "application/json")
    xhr.send(null)
  })
}
```

1 6 变量提升的原因与问题



变量提升的好处:

1. **提高性能**: 预编译时一次解析变量和函数声明, 分配栈空间, 避免每次执行重复解析
2. **容错性更好**: 即使先使用后声明也能正常执行

变量提升的问题:

```
// 问题1: 内层变量覆盖外层
var tmp = new Date();
function fn() {
  console.log(tmp); // undefined (不是外部 Date)
  if (false) {
    var tmp = 'hello'; // 提升到函数顶部
  }
}

// 问题2: 循环变量泄露为全局
for (var i = 0; i < 10; i++) { }
console.log(i) // 10 (i 变成全局变量)
```

ES6 的 `let / const` 通过块级作用域解决了这些问题。

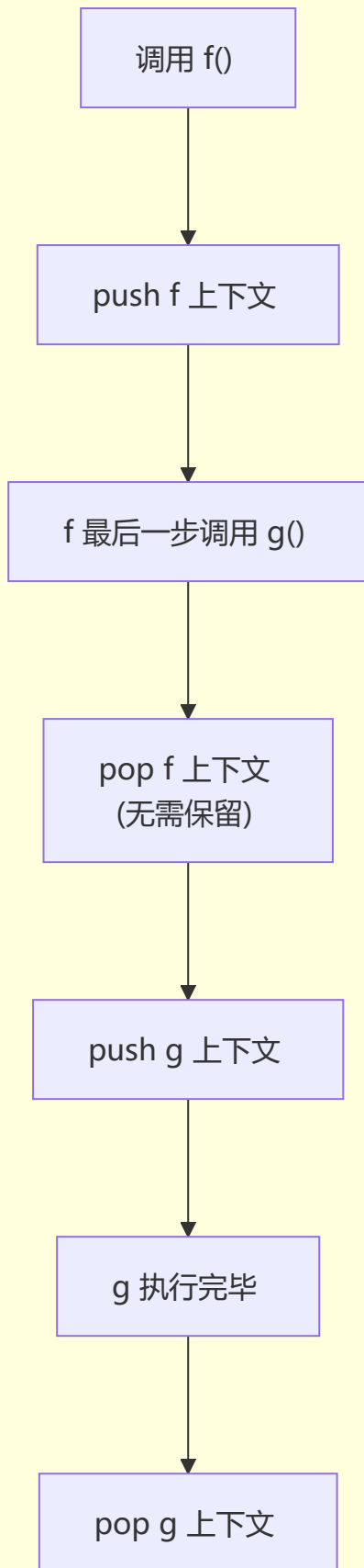
1 7 什么是尾调用?

尾调用 (Tail Call): 函数的最后一步是调用另一个函数。

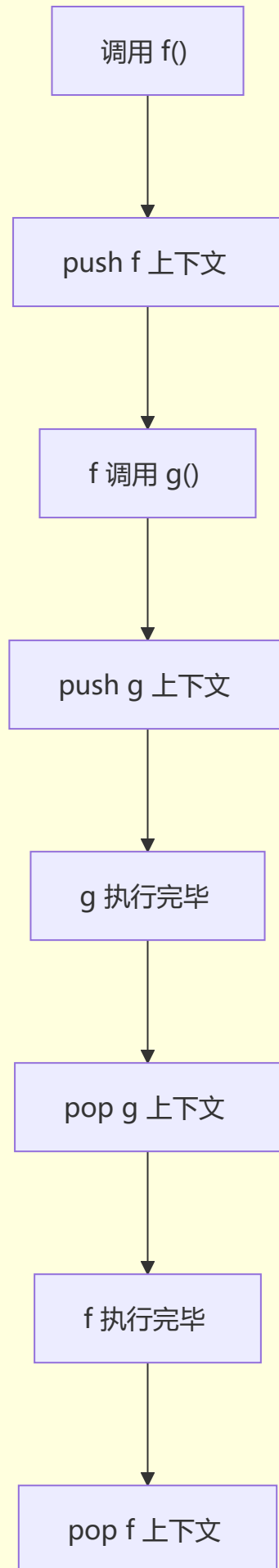
```
// 尾调用 ✅  
function f(x) {  
  return g(x)  
}  
  
// 不是尾调用 ❌  
function f(x) {  
  return g(x) + 1 // 还有加操作  
}  
  
function f(x) {  
  g(x) // 隐式返回 undefined, 不是函数最后一步的调用  
}
```

尾调用优化：由于是最后一步，当前函数的执行上下文不需要保留，可以直接复用，从而节省内存。ES6 的尾调用优化只在**严格模式**下开启。

尾调用优化



无尾调用优化



1 8 ES6 模块与 CommonJS 模块的异同

对比维度	ES6 Module	CommonJS
输出方式	值的引用（只读）	值的拷贝
加载方式	编译时加载（静态）	运行时加载（动态）
是否可修改导入值	✗（只读引用）	✓（可以重新赋值）
语法	<code>import / export</code>	<code>require() / module.exports</code>
树摇（Tree Shaking）	✓ 支持	✗ 不支持

```
// CommonJS - 值的拷贝
let count = 0
module.exports = { count, increment: () => count++ }
// 导入的是 count 的拷贝，导出后再变化不影响已导入的值

// ES6 Module - 值的引用
export let count = 0
export const increment = () => count++
// 导入的是对 count 的引用，始终同步
```

1 9 常见的 DOM 操作

```
// 获取节点
document.getElementById('id')
document.getElementsByTagName('div')
document.getElementsByClassName('cls')
document.querySelector('.cls')
document.querySelectorAll('div.cls')

// 创建节点
const div = document.createElement('div')
const text = document.createTextNode('hello')
const fragment = document.createDocumentFragment()

// 插入节点
parent.appendChild(child)
parent.insertBefore(newNode, referenceNode)

// 删除节点
parent.removeChild(child)
child.remove() // ES5+

// 修改节点
element.innerHTML = '<span>new</span>'
element.textContent = 'new text'
```

```
element.setAttribute('class', 'newClass')
element.style.color = 'red'

// 遍历节点
element.parentNode
element.childNodes
element.children
element.nextSibling
element.previousSibling
```

2 0 use strict 是什么?

严格模式 (Strict Mode) 是 ES5 引入的一种更严格的运行模式。

```
"use strict"; // 全局或函数级开启

// 主要区别:
// 1. 禁止 with 语句
with (obj) { x = 1 } // SyntaxError

// 2. 禁止 this 指向全局
function test() { console.log(this) }
test() // undefined (非严格模式为 window)

// 3. 禁止重复属性名
var obj = { a: 1, a: 2 } // SyntaxError

// 4. 禁止删除不可删除属性
delete Object.prototype // TypeError

// 5. 禁止八进制字面量
var x = 010 // SyntaxError

// 6. 必须声明变量
x = 1 // ReferenceError
```

2 1 如何判断对象属于某个类?

```
// 1. instanceof (检查原型链)
obj instanceof Constructor

// 2. constructor (可能被改写)
obj.constructor === Constructor

// 3. Object.prototype.toString (最可靠)
Object.prototype.toString.call(obj) // 如 [object Array]

// 4. isPrototypeOf
Constructor.prototype.isPrototypeOf(obj)
```

2 2 强类型 vs 弱类型语言

特性	强类型 (Java/C++)	弱类型 (JavaScript)
类型约束	变量类型固定，需强制转换	变量类型可隐式转换
示例	<code>int a = "1"</code> ❌	<code>var a = "1"</code> ✅
严谨性	高，编译期错误少	低，运行时易出错
灵活性	低	高

2 3 解释型 vs 编译型语言

特性	编译型 (C/C++/Go)	解释型 (JavaScript/Python)
执行时机	执行前编译成机器码	运行时逐行解释
执行效率	高	低
跨平台	差（需重新编译）	好（平台提供解释器）
开发速度	慢（需编译）	快（即时运行）

JavaScript 是**解释型语言**，但现代 JavaScript 引擎（V8）使用 **JIT (Just-In-Time) 编译** 技术，将热点代码编译为机器码，大幅提升性能。

2 4 for...in 和 for...of 的区别



```
const arr = ['a', 'b', 'c']
Array.prototype.customProp = 'prototype'

for (const key in arr) {
  console.log(key) // '0', '1', '2', 'customProp' (原型链上的也遍历了)
}

for (const val of arr) {
  console.log(val) // 'a', 'b', 'c' (只遍历自身元素)
}
```

2 5 使用 for...of 遍历对象

默认对象不可迭代，需要手动添加 `[Symbol.iterator]`：

```
// 方法1：手动实现迭代器
const obj = { a: 1, b: 2, c: 3 }
obj[Symbol.iterator] = function() {
```



```

const keys = Object.keys(this)
let count = 0
return {
  next: () => {
    if (count < keys.length) {
      return { value: this[keys[count++]], done: false }
    }
    return { value: undefined, done: true }
  }
}
}

// 方法2: 使用 Generator
obj[Symbol.iterator] = function*() {
  const keys = Object.keys(this)
  for (const k of keys) {
    yield [k, this[k]]
  }
}
}

```

2 6 ajax、axios、fetch 的区别

基于 XMLHttpRequest/Promise 第三方库 请求/响应拦截 自动 JSON 转换 支持跨域请求 支持浏览器缓存 支持 CORS	基于 Promise 原生 API (ES6) 返回 Promise 400/500 不 reject 默认不带 cookie 不支持浏览器缓存	基于 XMLHttpRequest 原生 API 跨域请求 数据格式
--	---	---

2 7 数组遍历方法对比

方法	返回值	是否改变原数组	特点
forEach()	undefined	✗	最基础遍历
map()	新数组	✗	映射转换
filter()	新数组	✗	过滤筛选
some()	boolean	✗	有一个满足即 true
every()	boolean	✗	全部满足才 true
find()	元素值	✗	返回第一个满足的元素
findIndex()	索引	✗	返回第一个满足的索引
reduce()	累计值	✗	归并计算
reduceRight()	累计值	✗	从右向左归并
flat()	新数组	✗	扁平化
flatMap()	新数组	✗	map + flat

2 8 forEach 和 map 的区别

对比	forEach	map
返回值	undefined	新数组
原始数组	可能被操作改变	不变
链式调用	✖	✔
用途	执行副作用	数据转换

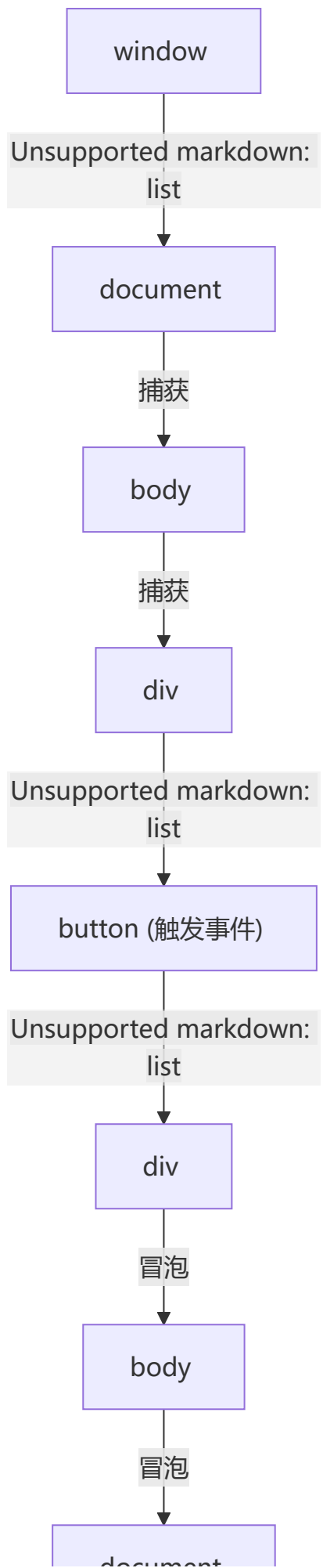
29. addEventListener() 的使用

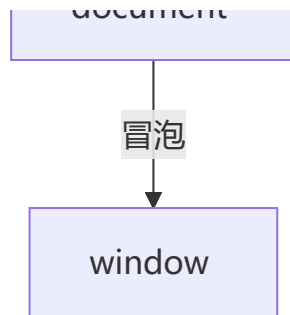
```
target.addEventListener(type, listener, options)
target.addEventListener(type, listener, useCapture)
```

options 参数:

```
{
  capture: false,    // 是否在捕获阶段触发
  once: false,       // 是否只触发一次后自动移除
  passive: false,    // 是否从不调用 preventDefault (提升滚动性能)
  signal: null       // AbortSignal, 用于移除监听器
}
```

事件流三阶段:

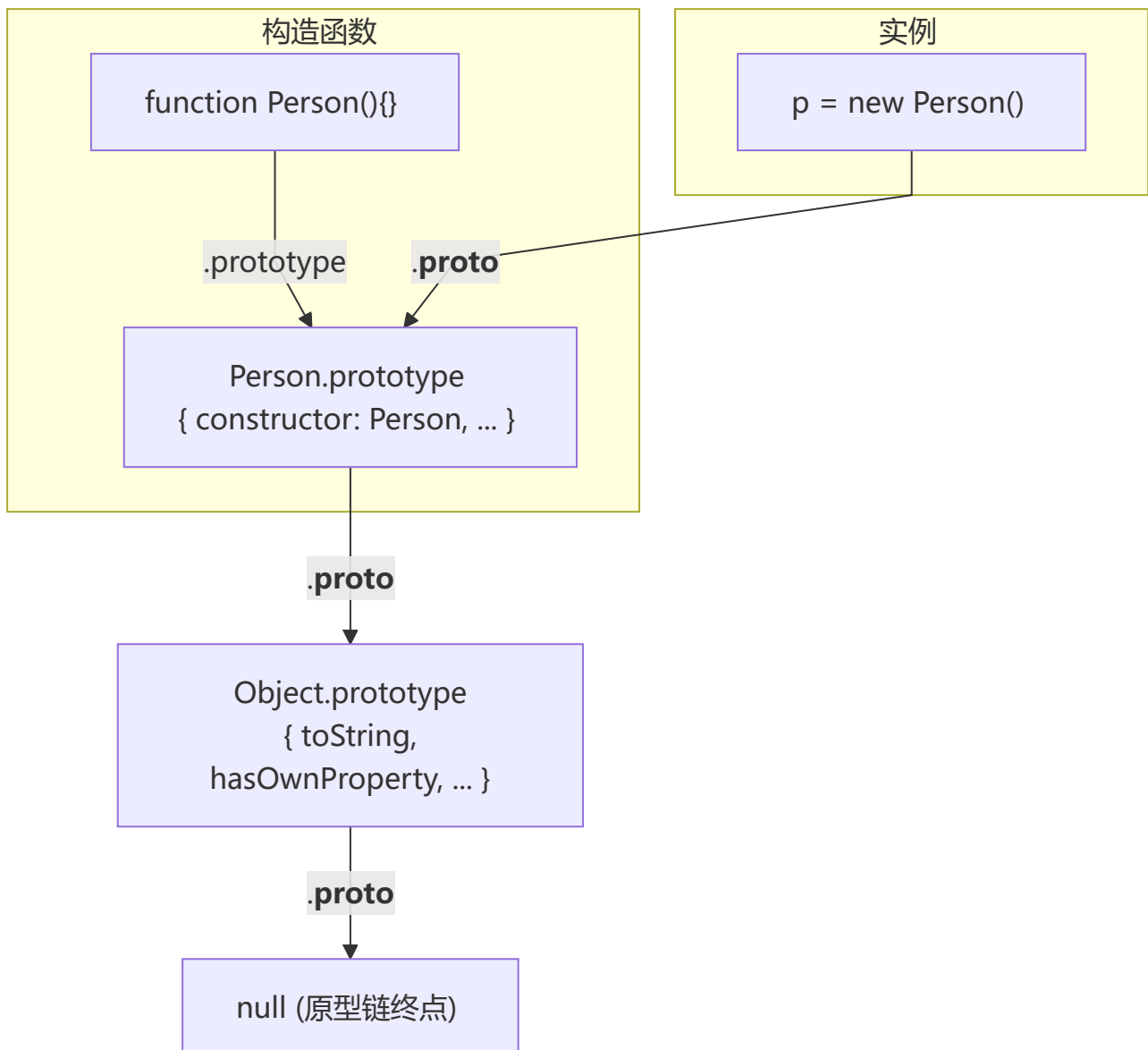




🔗 四、原型与原型链

1 对原型、原型链的理解

💡 要点：每个函数有 `prototype`（显式原型），每个对象有 `__proto__`（隐式原型）。访问属性时沿 `__proto__` 链向上查找直到 `null`。原型链终点是 `Object.prototype.__proto__` → `null`。



核心概念：

1. 每个**构造函数**都有一个 `prototype` 属性，指向原型对象
2. 每个**对象**都有一个 `__proto__` 属性（也称隐式原型），指向其构造函数的 `prototype`
3. 访问对象的属性时，若自身不存在则沿着 `__proto__` 链向上查找，直到找到或到达 `null`
4. 原型链终点是 `Object.prototype.__proto__` → `null`

特点：JavaScript 对象通过引用传递，修改原型会影响所有相关对象。

2 原型修改与重写

```
function Person(name) {  
  this.name = name  
}  
  
// 修改原型（添加属性/方法）  
Person.prototype.getName = function() { return this.name }  
var p1 = new Person('hello')  
console.log(p1.__proto__ === Person.prototype)           // true  
console.log(p1.__proto__ === p1.constructor.prototype)   // true  
  
// 重写原型（整个替换）  
Person.prototype = {  
  getName: function() { return this.name }  
}  
var p2 = new Person('hello')  
console.log(p2.__proto__ === Person.prototype)           // true  
console.log(p2.__proto__ === p2.constructor.prototype)   // false!  
// 原因是 Person.prototype = {} 切断了 constructor 引用
```

修复 constructor 引用：

```
Person.prototype = {  
  constructor: Person, // 手动指回  
  getName: function() { return this.name }  
}
```

3 原型链指向

```
p.__proto__           // Person.prototype  
Person.prototype.__proto__ // Object.prototype  
p.__proto__.__proto__ // Object.prototype  
p.__proto__.constructor.prototype.__proto__ // Object.prototype  
Person.prototype.constructor // Person  
Person.prototype.constructor.prototype // Person.prototype
```

4 原型链的终点是什么？

原型链的终点是 `null`。

```
object.prototype.__proto__ // null
```

因为 `Object.prototype` 是原型链的顶端，它的原型是 `null`，没有任何属性和方法。

5 如何获得对象非原型链上的属性？

使用 `hasOwnProperty()` 方法判断属性是否是对象自身的（而不是原型链上的）：

```
function iterate(obj) {  
  var res = []  
  for (var key in obj) {  
    if (obj.hasOwnProperty(key)) {  
      res.push(key + ': ' + obj[key])  
    }  
  }  
  return res  
}
```

`in` 操作符 vs `hasOwnProperty`：

- `'key' in obj`：检查自身和原型链上的所有属性
- `obj.hasOwnProperty('key')`：只检查自身属性

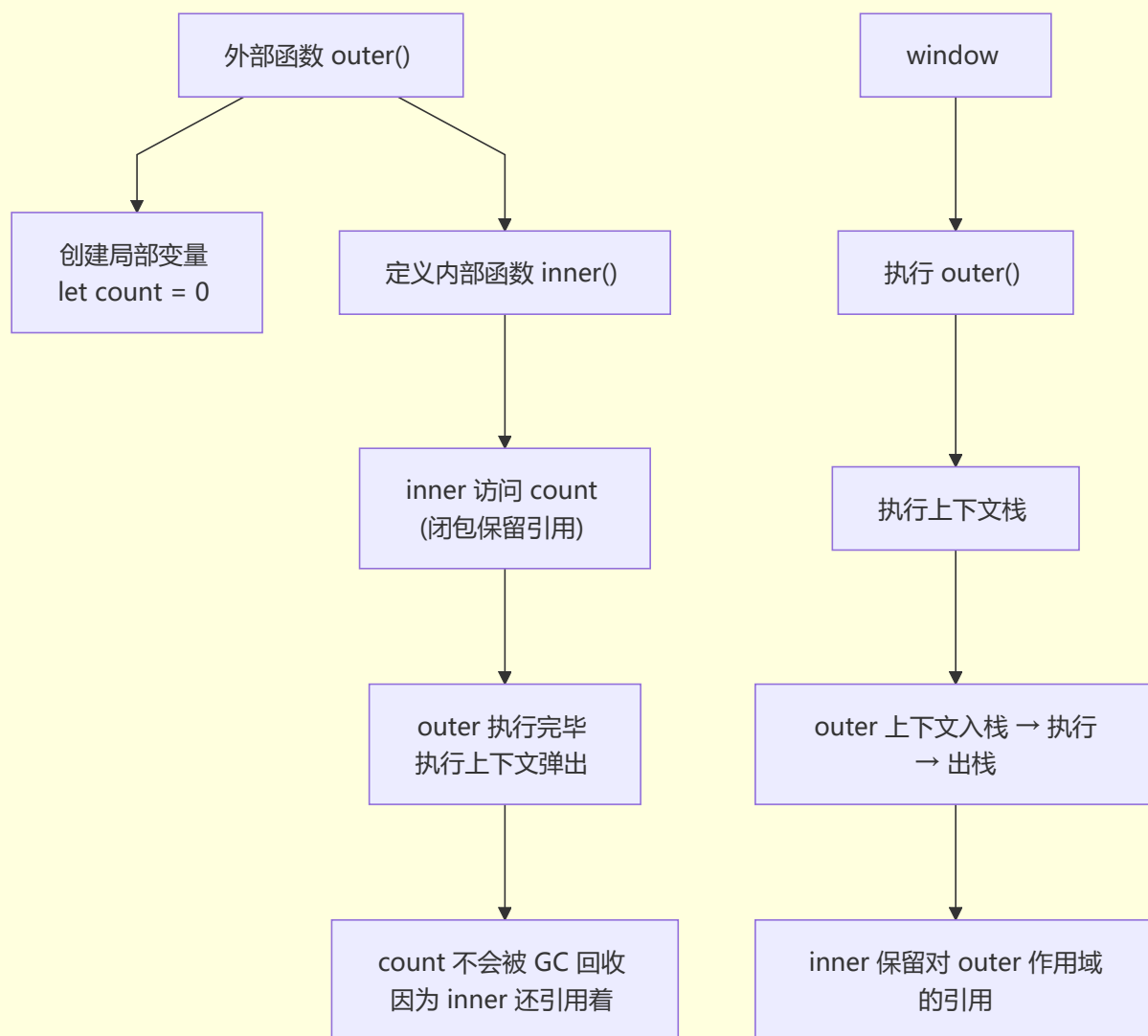
🎯 五、执行上下文 / 作用域链 / 闭包

1 对闭包的理解

💡 **要点：**闭包是指内部函数可以访问外部函数作用域中变量的能力。即使外部函数执行完毕，其变量对象仍被内部函数引用而无法被 GC 回收。

闭包 (Closure)： 一个函数内定义了另一个函数，内部函数可以访问外部函数作用域中的变量。

闭包工作原理



闭包的用途:

1. 创建私有变量:

```
function createCounter() {
  let count = 0
  return {
    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count
  }
}

const counter = createCounter()
counter.increment() // 1
counter.increment() // 2
counter.getCount()  // 2
```

2. 函数柯里化:

```
function multiply(a) {  
  return function(b) {  
    return a * b  
  }  
}  
  
const double = multiply(2)  
double(5) // 10
```

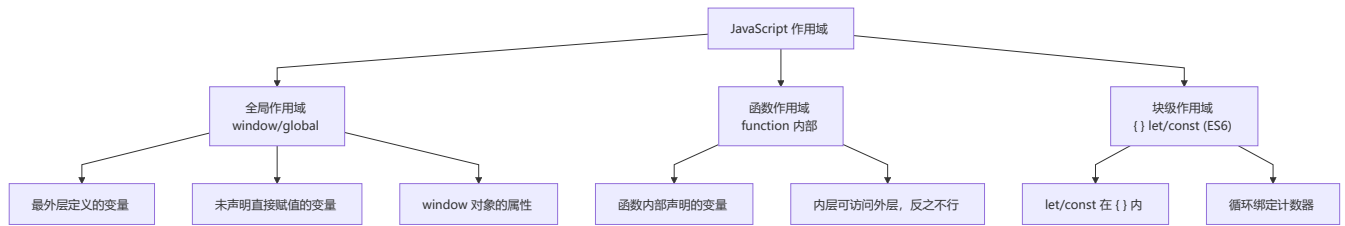
3. 经典面试题——循环中的闭包：

```
// 问题：输出 6 个 6  
for (var i = 1; i <= 5; i++) {  
  setTimeout(function timer() {  
    console.log(i)  
  }, i * 1000)  
}  
  
// 解决方案1: IIFE 闭包  
for (var i = 1; i <= 5; i++) {  
  (function(j) {  
    setTimeout(function timer() {  
      console.log(j)  
    }, j * 1000)  
  })(i)  
}  
  
// 解决方案2: setTimeout 第三个参数  
for (var i = 1; i <= 5; i++) {  
  setTimeout(function timer(j) {  
    console.log(j)  
  }, i * 1000, i)  
}  
  
// 解决方案3: let 块级作用域（推荐）  
for (let i = 1; i <= 5; i++) {  
  setTimeout(function timer() {  
    console.log(i)  
  }, i * 1000)  
}
```

⚠ **注意：**不合理使用闭包会导致内存泄漏，因为被引用的外部变量无法被 GC 回收。使用闭包时要注意及时释放不再需要的引用（如将变量置为 `null`）。

2 对作用域与作用域链的理解

作用域 (Scope) 的类型：



作用域链 (Scope Chain) :

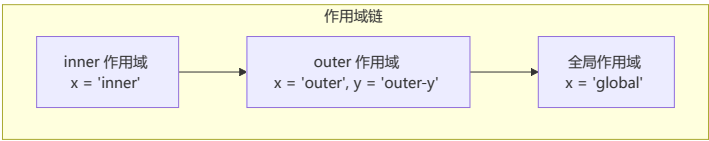
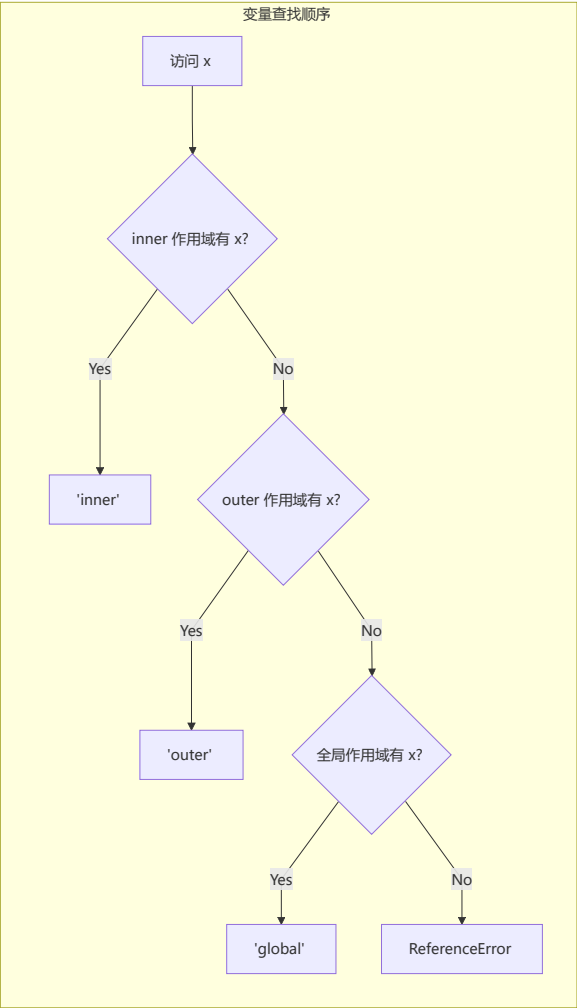
```
const x = 'global'

function outer() {
  const x = 'outer'
  const y = 'outer-y'

  function inner() {
    const x = 'inner'
    console.log(x) // 'inner' (自己的 x)
    console.log(y) // 'outer-y' (从 outer 作用域查找)
  }

  console.log(x) // 'outer' (自己的 x)
  inner()
}

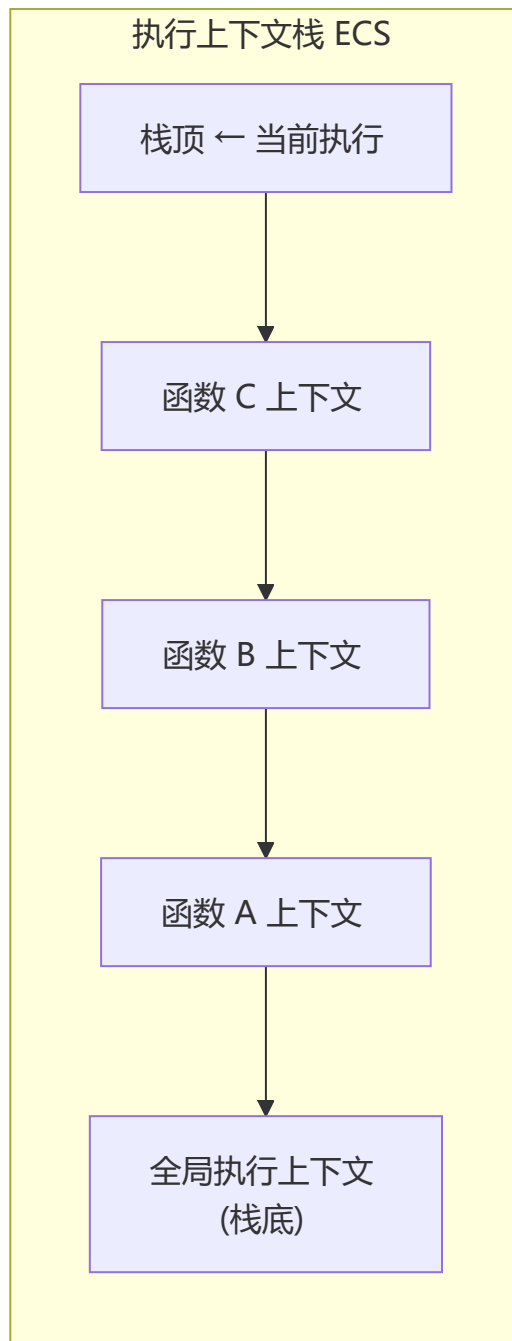
outer()
```



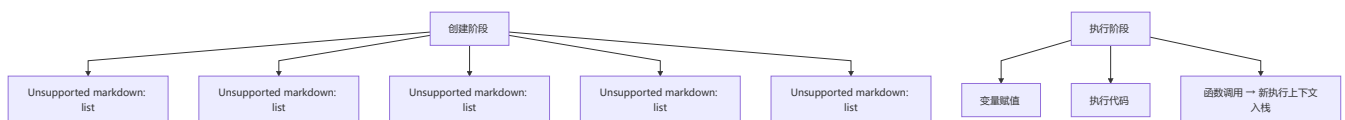
作用域链的实质： 一个指向变量对象的指针列表。前端始终是当前执行上下文的变量对象，末端是全局执行上下文的变量对象。

3 对执行上下文的理解

执行上下文类型：



创建执行上下文的两个阶段：



全局执行上下文：

- 任何不在函数内部的代码
- 创建全局 `window` 对象
- `this` 指向全局对象
- 程序中只有一个全局执行上下文

函数执行上下文：

- 每次函数调用时创建
- 可以有任意多个
- 比全局上下文多 `this`、`arguments` 和函数参数

🎯 六、this / call / apply / bind

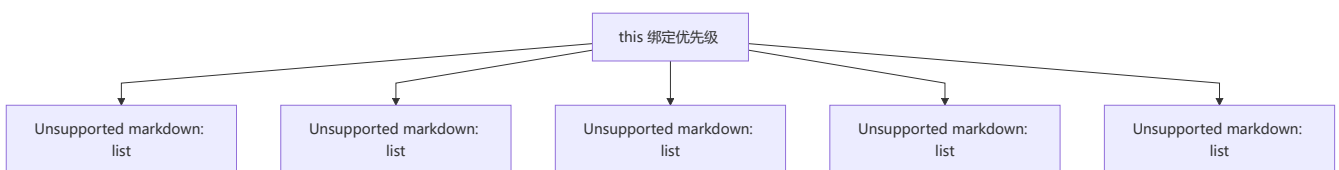
1 对 this 的理解

💡 要点: `this` 的指向遵循"谁调用指向谁"的原则。箭头函数不绑定 `this`，而是继承外层作用域的 `this`。

优先级: `new` > `call/apply/bind` > 隐式绑定 > 默认绑定。

`this` 是执行上下文中的一个属性，指向最后一次调用这个方法的对象。

四种调用模式（优先级从高到低）：



示例：

```
// 1. 函数调用模式 (this → window/global)
function test() { console.log(this) }
test() // window (非严格模式) / undefined (严格模式)

// 2. 方法调用模式 (this → 调用对象)
const obj = { name: 'obj', test: function() { console.log(this) } }
obj.test() // obj

// 3. 构造器调用模式 (this → 新创建对象)
function Person(name) { this.name = name }
const p = new Person('Tom')
console.log(p.name) // 'Tom'

// 4. call/apply/bind (this → 指定对象)
function greet() { console.log(this.name) }
greet.call({ name: 'call' }) // 'call'
greet.apply({ name: 'apply' }) // 'apply'
greet.bind({ name: 'bind' })() // 'bind'
```

2 call 和 apply 的区别

相同点: 都用于改变函数执行时的 `this` 指向，立即调用函数。

不同点: 参数传递形式不同。

方法	第二个参数	使用场景
<code>call(thisArg, arg1, arg2, ...)</code>	参数列表	参数数量固定时
<code>apply(thisArg, [argsArray])</code>	数组或类数组	参数数量不固定或已有数组时

```
function sum(a, b, c) { return a + b + c }
```

```
sum.call(null, 1, 2, 3) // 6
```

```
sum.apply(null, [1, 2, 3]) // 6
```

```
// 典型应用：借用数组方法
```

```
Math.max.apply(null, [1, 3, 2]) // 3
```

```
// ES6 写法: Math.max(...[1, 3, 2])
```

3 实现 call、apply、bind

call 实现:

```
Function.prototype.myCall = function(context) {
  if (typeof this !== "function") throw new TypeError("Error")
  const args = Array.from(arguments).slice(1)
  context = context || window
  const fnSymbol = Symbol() // 避免属性名冲突
  context[fnSymbol] = this
  const result = context[fnSymbol](...args)
  delete context[fnSymbol]
  return result
}
```

apply 实现:

```
Function.prototype.myApply = function(context, argsArr) {
  if (typeof this !== "function") throw new TypeError("Error")
  context = context || window
  const fnSymbol = Symbol()
  context[fnSymbol] = this
  const result = argsArr
    ? context[fnSymbol](...argsArr)
    : context[fnSymbol]()
  delete context[fnSymbol]
  return result
}
```

bind 实现:

```
Function.prototype.myBind = function(context, ...bindArgs) {  
  if (typeof this !== "function") throw new TypeError("Error")  
  const fn = this  
  return function boundFn(...callArgs) {  
    return fn.apply(  
      this instanceof boundFn ? this : context,  
      bindArgs.concat(callArgs)  
    )  
  }  
}
```



七、异步编程

异步编程的实现方式

JavaScript 异步编程演进

Unsupported markdown:
list



问题: 回调地狱



Unsupported markdown:
list



问题: then 链仍冗长



Unsupported markdown:
list



问题: 需手动执行

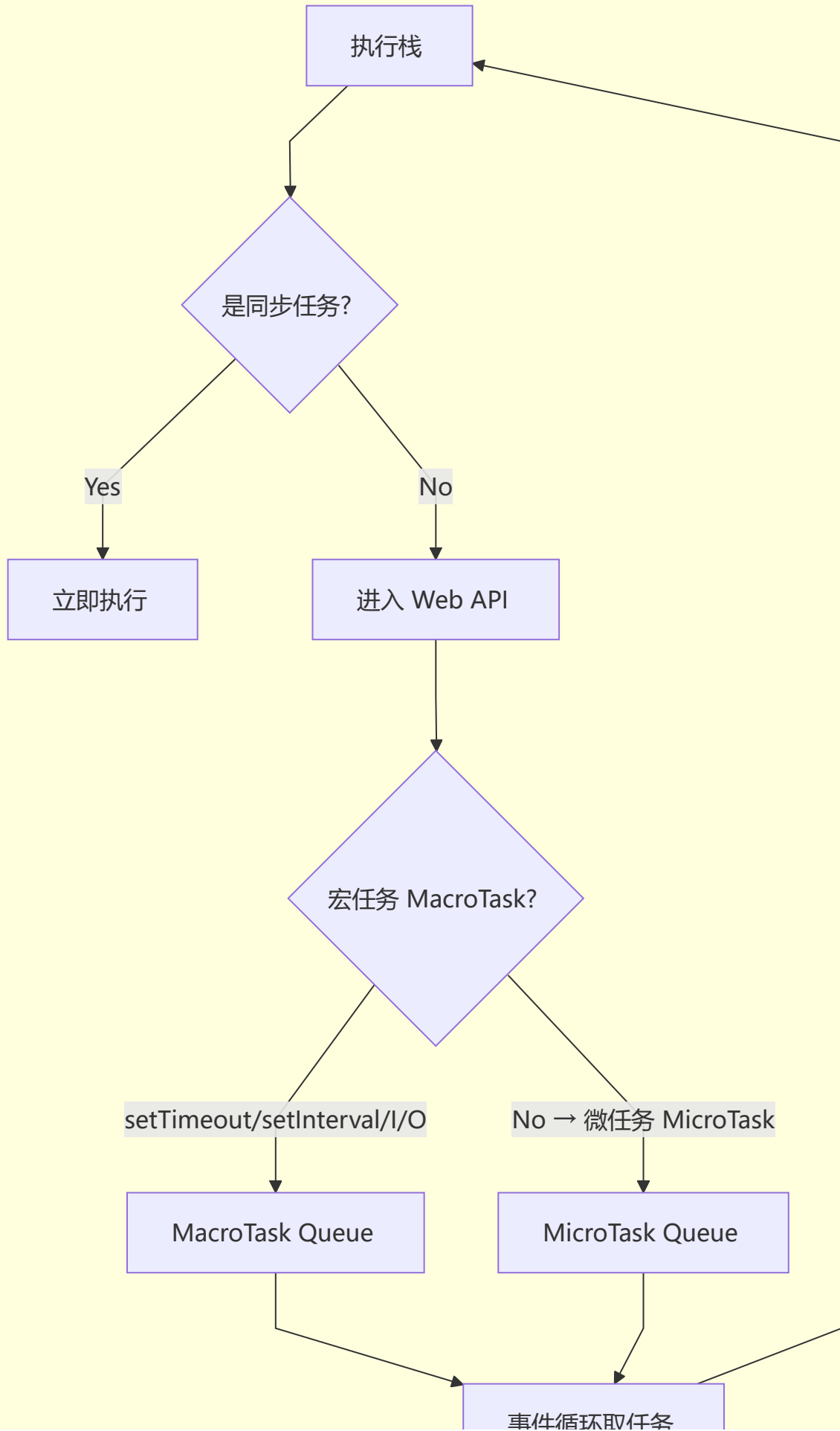


Unsupported markdown:
list

方式	优点	缺点
回调函数	简单、通用	回调地狱、耦合高
Promise	链式调用、错误处理好	then 链仍不够直观
Generator	同步风格写法	需要执行器
async/await	同步风格、自动执行	不能并行（需配合 Promise.all）

2 setTimeout、Promise、Async/Await 的区别

事件循环 Event Loop



执行顺序示例：

```
console.log('script start') // 1. 同步

setTimeout(() => {
  console.log('settimeout') // 4. 宏任务
}, 0)

Promise.resolve()
  .then(() => console.log('promise1')) // 3. 微任务
  .then(() => console.log('promise2')) // 5. 第二个微任务

console.log('script end') // 2. 同步

// 输出: script start → script end → promise1 → promise2 → settimeout
```

async/await 执行顺序详解：

```
async function async1() {
  console.log('async1 start') // 2
  await async2() // await 让出线程
  console.log('async1 end') // 5 (微任务)
}

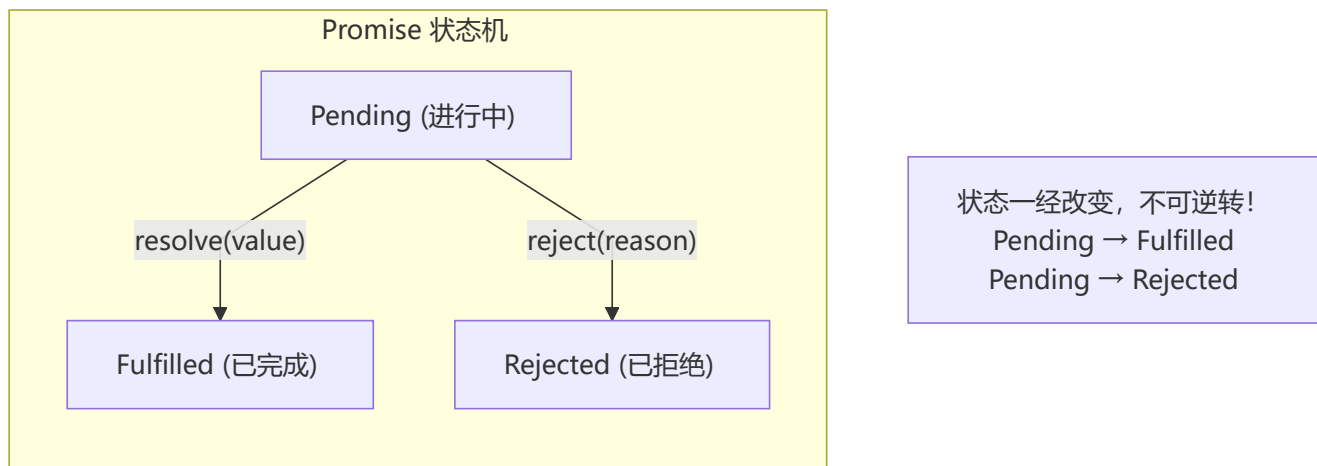
async function async2() {
  console.log('async2') // 3
}

console.log('script start') // 1
async1()
console.log('script end') // 4

// 输出: script start → async1 start → async2 → script end → async1 end
```

3 对 Promise 的理解

💡 **要点：**Promise 是一个状态机，从 Pending 可以变为 Fulfilled 或 Rejected，且状态一旦改变就不可逆转。构造函数中的代码是**立即执行的**，`.then()` 中的回调是**微任务**。



Promise 的三个状态：

- **Pending (进行中)**：初始状态，异步操作尚未完成
- **Fulfilled (已成功)**：异步操作成功完成，调用 `resolve()`
- **Rejected (已失败)**：异步操作失败，调用 `reject()`

特点：

1. 状态不受外界影响，只能由异步操作结果决定
2. 状态一旦改变就不会再变
3. 构造函数中的代码是**立即执行的**

4 Promise 的基本用法

创建 Promise：

```
const promise = new Promise((resolve, reject) => {  
  // 异步操作  
  if (success) {  
    resolve(value)    // → fulfilled  
  } else {  
    reject(error)     // → rejected  
  }  
})
```

快捷方式：

```
Promise.resolve(42)    // 等价于 new Promise(r => r(42))  
Promise.reject('err')  // 等价于 new Promise( (_, r) => r('err'))
```

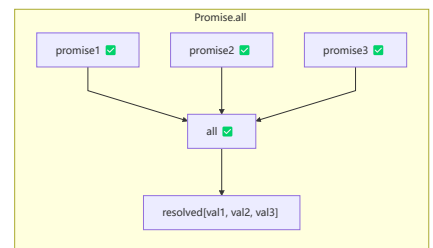
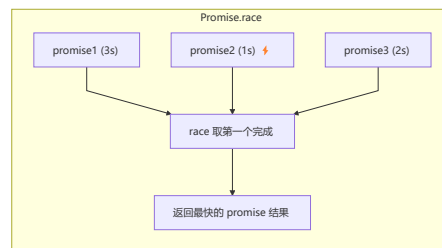
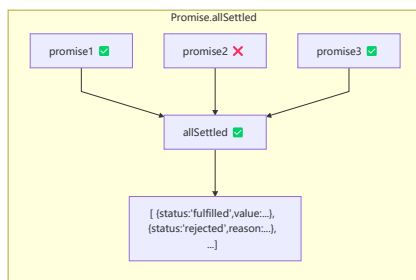
实例方法：

```
// then: 注册回调, 返回新 Promise
promise.then(
  value => { /* fulfilled */ },
  reason => { /* rejected */ }
)

// catch: rejected 回调 + 处理 then 中的异常
promise.catch(error => { /* 处理错误 */ })

// finally: 无论成功失败都执行
promise.finally(() => { /* 清理操作 */ })
```

静态方法:



```
Promise.all([p1, p2, p3]) // 全部成功才成功, 一个失败则失败
Promise.race([p1, p2, p3]) // 第一个完成的结果 (无论成功失败)
Promise.allSettled([p1, p2]) // 等所有完成, 返回每个的状态
Promise.any([p1, p2]) // 第一个成功的 (忽略失败)
```

5 Promise 解决了什么问题?

解决"回调地狱" (Callback Hell) :

```
// 回调地狱 (嵌套金字塔)
fs.readFile('./a.txt', function(err, data) {
  fs.readFile(data, function(err, data) {
    fs.readFile(data, function(err, data) {
      console.log(data)
    })
  })
})

// Promise 扁平链式调用
read('./a.txt')
  .then(data => read(data))
  .then(data => read(data))
  .then(data => console.log(data))
  .catch(err => console.error(err))
```

6 Promise.all 和 Promise.race 的使用场景

Promise.all 适用场景：

- 多个不相关的请求需要全部完成后处理
- 并行请求且需要保持结果顺序

```
// 并行请求用户信息
const [user, posts, friends] = await Promise.all([
  fetch('/user'),
  fetch('/posts'),
  fetch('/friends')
])
```

Promise.race 适用场景：

- 超时控制
- 多个数据源取最快响应

```
// 超时控制
const timeout = (ms) => new Promise((_, reject) =>
  setTimeout(() => reject(new Error('timeout')), ms)
)

Promise.race([
  fetch('/data'),
  timeout(5000)
]).then(data => console.log(data))
  .catch(err => console.log('请求超时'))
```

7 对 async/await 的理解

async/await 是 **Generator + Promise** 的语法糖。

```
async function test() {
  return 'hello'
}

// 等价于
function test() {
  return Promise.resolve('hello')
}

// async 函数返回 Promise
test().then(v => console.log(v)) // 'hello'
```

8 await 到底在等什么?

```
async function test() {  
  const v1 = await '直接量'           // v1 = '直接量' (不是 Promise 直接返回值)  
  const v2 = await Promise.resolve(42) // v2 = 42 (Promise 则等 resolve)  
  return [v1, v2]  
}  
  
test().then(console.log) // ['直接量', 42]
```

await 的工作原理:

1. 如果 await 后面不是 Promise → 直接返回值
2. 如果 await 后面是 Promise → 暂停 async 函数执行, 等待 Promise resolve, 然后继续执行

9 async/await 的优势

```
// Promise 链式  
function doIt() {  
  return step1(300)  
    .then(time2 => step2(time2))  
    .then(time3 => step3(time3))  
    .then(result => console.log(result))  
}  
  
// async/await (更清晰)  
async function doIt() {  
  const time1 = 300  
  const time2 = await step1(time1)  
  const time3 = await step2(time2)  
  const result = await step3(time3)  
  console.log(result)  
}
```

对比	Promise	async/await
代码风格	链式	同步风格
中间值传递	需要嵌套/传参	直接赋值
错误处理	<code>.catch()</code>	<code>try/catch</code>
调试	困难 (不能步进 then)	友好 (像同步代码)

10 async/await 如何捕获异常

```
async function fn() {  
  try {  
    const data = await fetch('/api')  
    // 处理 data  
  } catch (error) {  

```

```

    // 网络错误 OR 返回的 rejected Promise
    console.error(error)
  }
}

// 全局捕获
async function fn2() {
  const data = await fetch('/api').catch(err => {
    console.error(err)
    return null // 返回默认值
  })
  if (data) { /* 处理 */ }
}

```



八、面向对象

1 对象创建的方式

模式	优点	缺点
字面量	简单直接	不适用于大量相似对象
工厂模式	封装复用代码	不能识别对象类型
构造函数模式	可识别类型 (instanceof)	方法不能复用 (每次创建新函数)
原型模式	方法复用	引用类型属性共享问题、不能传参
组合模式	构造函数初始化+原型复用方法	封装性不够好
动态原型模式	封装性更好	-
寄生构造函数模式	扩展已有类型	不能识别类型

最佳实践：组合使用构造函数和原型

```

function Person(name, age) {
  this.name = name // 实例属性 (每个实例独有)
  this.age = age
}

Person.prototype.sayName = function() { // 共享方法
  console.log(this.name)
}

```

2 对象继承的方式

继承方式	原理	优点	缺点
原型链继承	<code>Child.prototype = new Parent()</code>	简单	引用类型共享、不能传参

继承方式	原理	优点	缺点
借用构造函数	<code>Parent.call(this)</code>	可传参、独有属性	方法不能复用
组合继承	原型链 + 借用构造函数	两者优点	调用两次父构造函数
原型式继承	<code>Object.create()</code>	简洁	引用类型共享
寄生式继承	增强对象	可添加额外属性	方法不能复用
寄生组合继承	借用构造函数 + 原型副本	最理想	-

最佳实践：寄生组合继承

```
function Parent(name) {
  this.name = name
  this.colors = ['red', 'blue']
}

Parent.prototype.sayName = function() {
  console.log(this.name)
}

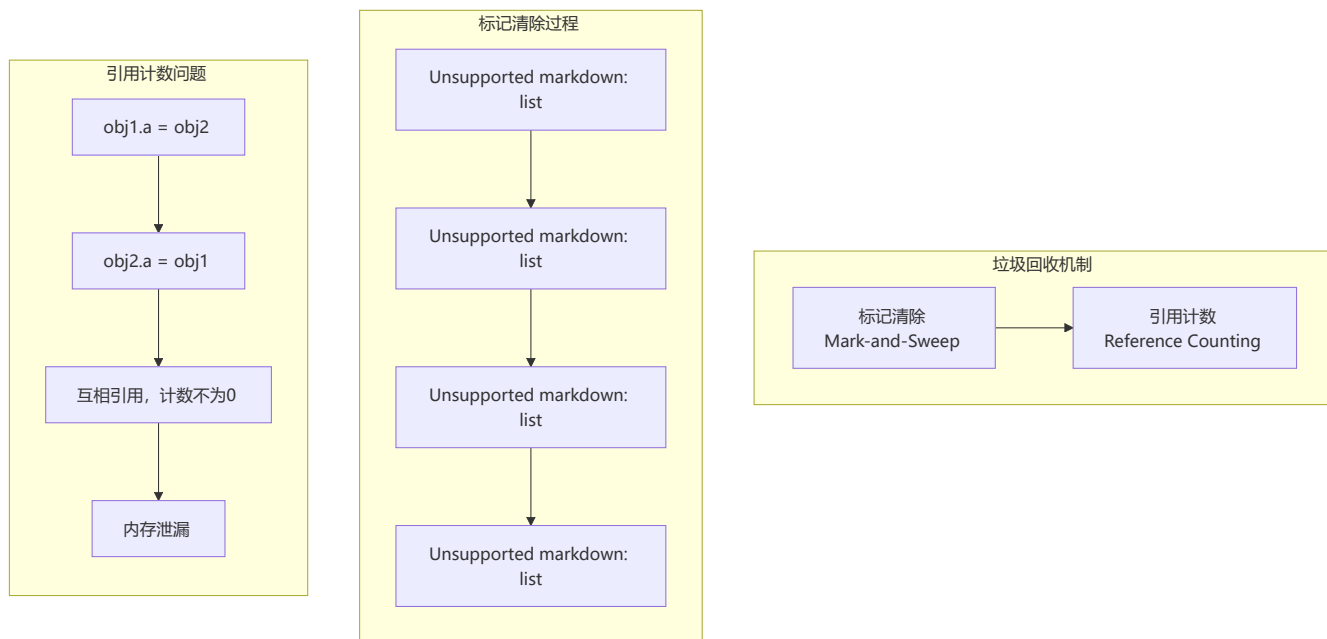
function Child(name, age) {
  Parent.call(this, name) // 继承实例属性
  this.age = age
}

// 关键：使用父类原型副本来替代父类实例
Child.prototype = Object.create(Parent.prototype)
Child.prototype.constructor = Child

Child.prototype.sayAge = function() {
  console.log(this.age)
}
```

九、垃圾回收与内存泄漏

1 浏览器的垃圾回收机制



现代浏览器使用"标记清除"算法，分为新生代（Scavenge）和老生代（Mark-Sweep/Mark-Compact）：

- **新生代：** 空间小、存活时间短的对象（局部变量），使用 Scavenge 算法
- **老生代：** 空间大、存活时间长的对象（全局变量、闭包），使用 Mark-Sweep + Mark-Compact

2 哪些情况会导致内存泄漏？

```
// 1. 意外的全局变量
function foo() {
  bar = 'global' // 未声明 → 全局变量
}

// 2. 遗忘的定时器
const data = { /* 大量数据 */ }
setInterval(() => {
  // 使用 data, 但未清理定时器
}, 1000)
// clearInterval(timer) // 忘记清理

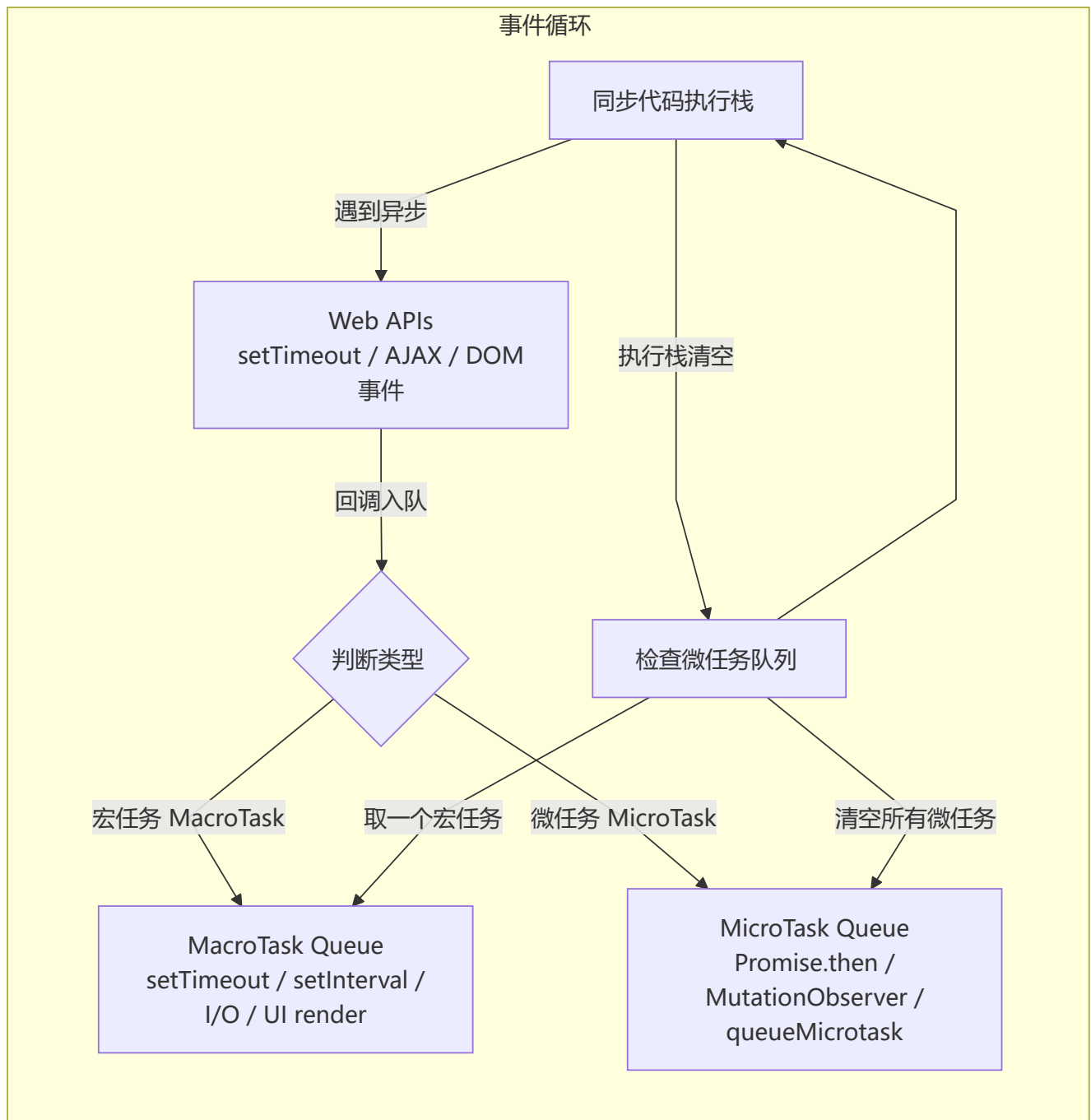
// 3. 脱离 DOM 的引用
const button = document.getElementById('btn')
document.body.removeChild(button)
// button 变量仍持有引用 → DOM 无法被 GC

// 4. 闭包不合理使用
function outer() {
  const largeData = new Array(1000000)
  return function inner() {
    // 引用 largeData, 外部持有 inner 时 largeData 无法释放
  }
}
```

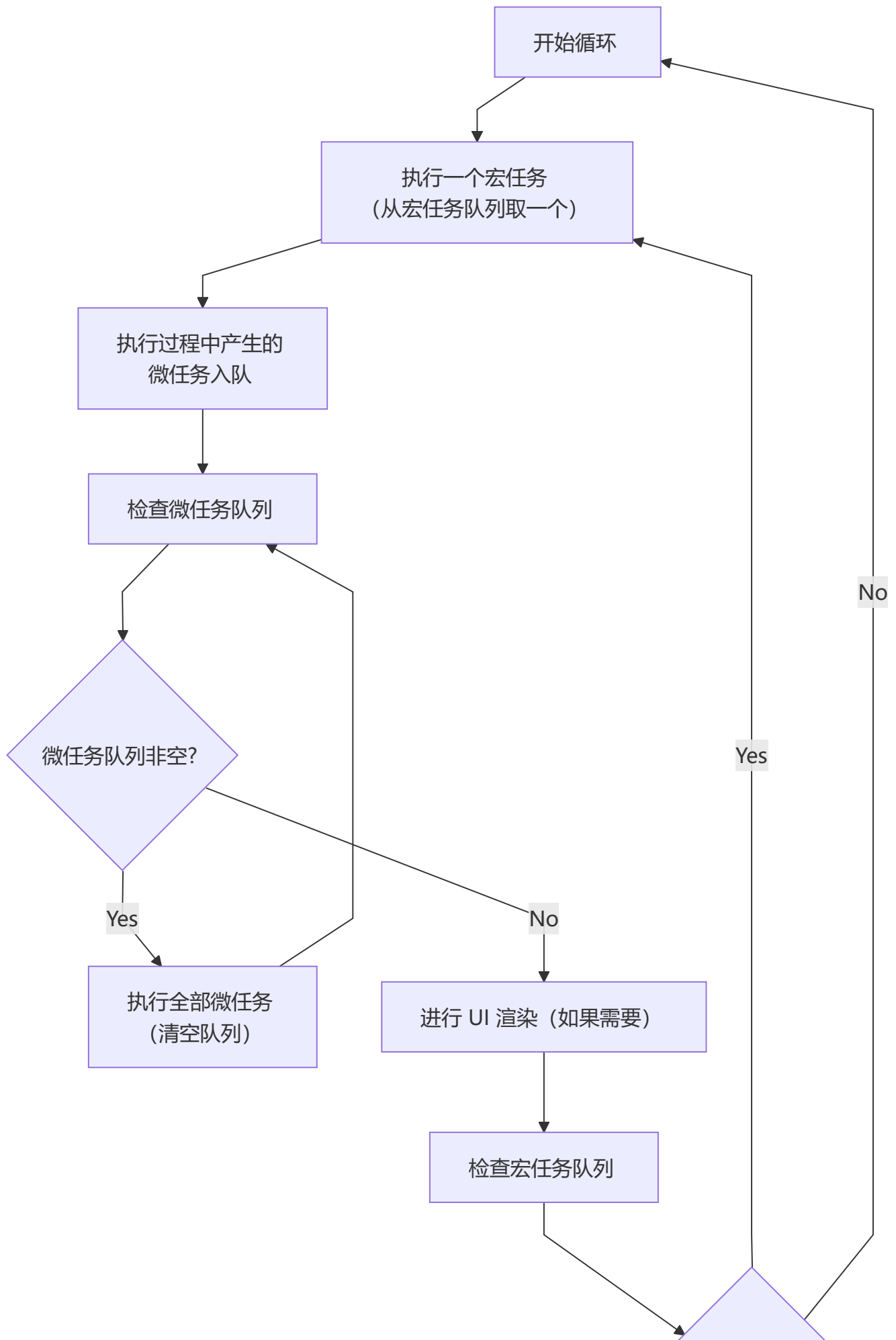
泄漏原因	解决方案
意外全局变量	使用严格模式 <code>'use strict'</code>
遗忘定时器	及时 <code>clearInterval</code> / <code>clearTimeout</code>
DOM 引用	删除 DOM 时同时清除引用
闭包	用完后置为 <code>null</code> 释放
事件监听器	<code>removeEventListener</code>
WeakMap/WeakSet	使用 WeakMap 替代 Map（键为对象时）

十、事件循环（Event Loop）详解

💡 **要点：** JS 是单线程语言，通过事件循环实现异步。执行顺序：同步代码 → 微任务（`Promise.then`/`MutationObserver`）→ 宏任务（`setTimeout`/`setInterval`/`I/O`）。每取一个宏任务执行，清空所有微任务，再进行 UI 渲染。



事件循环规则：



宏任务队列非空?

任务优先级:

1. **同步代码** (当前执行栈)
2. **微任务** (Promise.then/catch/finally、MutationObserver、queueMicrotask)
3. **宏任务** (setTimeout、setInterval、setImmediate、I/O、UI render)

经典面试题:

```
console.log('1') // 同步: 1

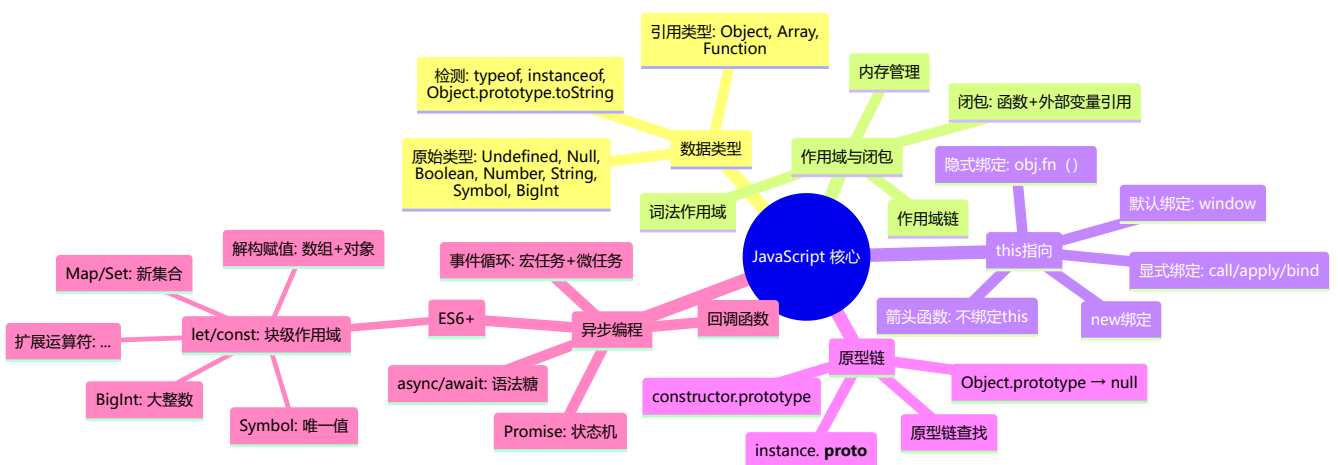
setTimeout(() => {
  console.log('2') // 宏任务
}, 0)

Promise.resolve().then(() => {
  console.log('3') // 微任务
})

console.log('4') // 同步: 4

// 输出顺序: 1 → 4 → 3 → 2
```

总结



🌟 十一、现代JavaScript新特性

1 可选链操作符 ?.

语法和用法

可选链 `?.` 允许读取位于连接对象链深处的属性，而不必明确验证链中的每个引用是否有效。如果某个环节是 `null` 或 `undefined`，表达式会短路返回 `undefined`。

```

const obj = {
  user: {
    address: {
      street: '长安街'
    }
  }
}

// 传统写法：层层判空
const street1 = obj && obj.user && obj.user.address && obj.user.address.street

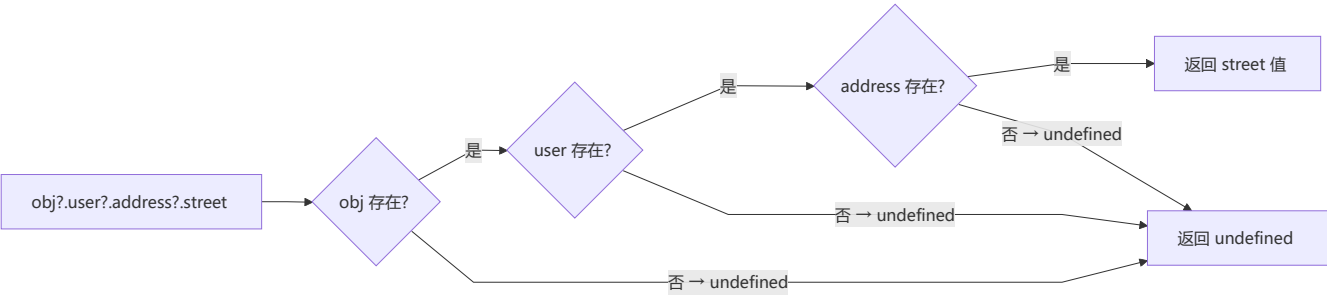
// 可选链写法
const street2 = obj?.user?.address?.street

// 函数调用
const result = obj?.method?.() // 如果 method 不存在返回 undefined

```

与 && 的对比

对比项	传统 &&	可选链 ?.
语法简洁度	嵌套多层时代码冗余	链式调用，语义清晰
短路行为	遇到 falsy 值即停止	仅对 null/undefined 短路
函数调用	<code>obj && obj.fn()</code>	<code>obj?.fn?().()</code>
表达式支持	支持任意表达式	仅支持属性访问和函数调用



实战: 深层嵌套属性安全读取

```
// API 返回的深层嵌套数据
const response = {
  data: {
    articles: [
      { title: 'JS 新特性', author: { name: '张三', email: 'zhang@example.com' } }
    ]
  }
}

// 安全读取第一篇文章的作者邮箱
const email = response?.data?.articles?.[0]?.author?.email ?? '未知'

// 动态属性名
const key = 'articles'
const firstTitle = response?.data?.[key]?.[0]?.title
```

2 空值合并操作符 ??

语法和用法

空值合并运算符 `??` 是一个逻辑运算符，当左侧的操作数为 `null` 或 `undefined` 时，返回其右侧操作数，否则返回左侧操作数。

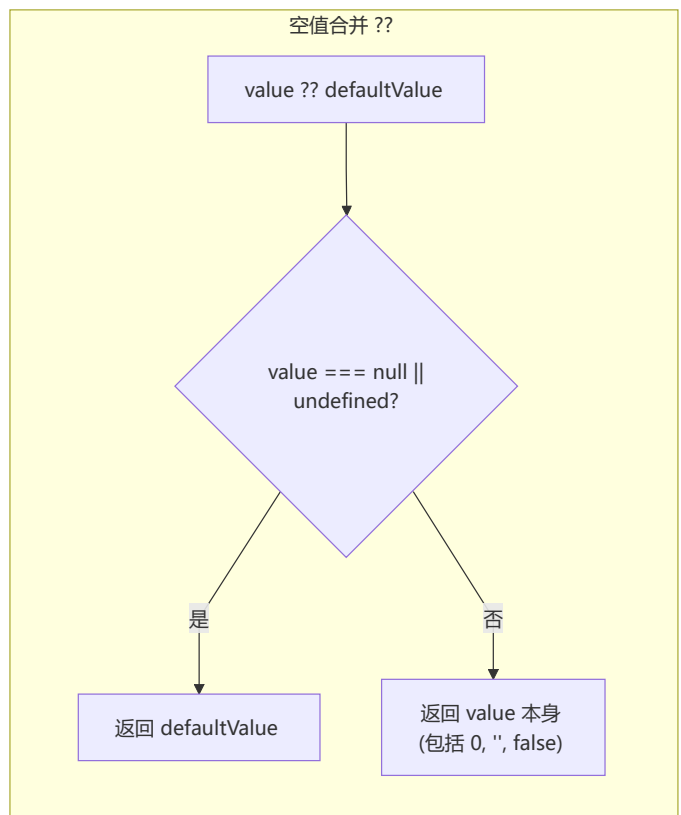
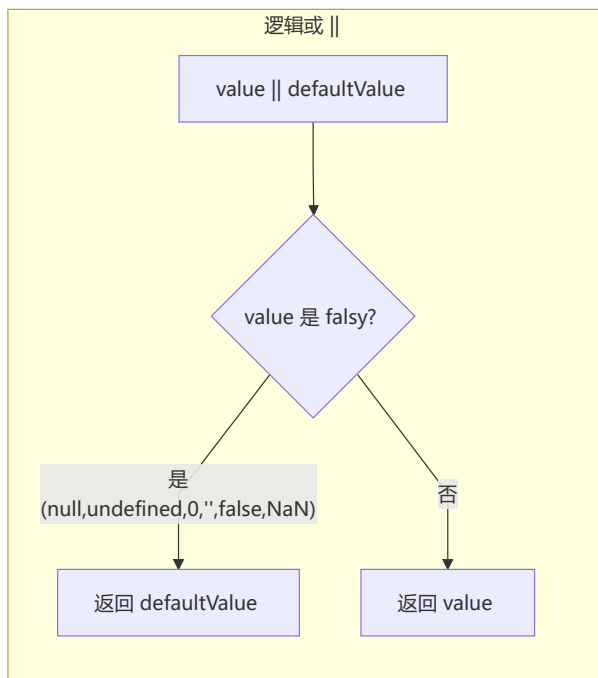
```
const foo = null ?? '默认值'    // '默认值'
const bar = 0 ?? '默认值'       // 0
const baz = '' ?? '默认值'      // ''
const qux = false ?? '默认值'   // false
```

与 `||` 的区别

`||` 对所有 falsy 值 (`0`、`''`、`false`、`null`、`undefined`、`NaN`) 都会取右侧值，而 `??` 仅对 `null` 和 `undefined`。

```
const count = 0
console.log(count || 100)    // 100 (0 被当作 falsy)
console.log(count ?? 100)    // 0 (0 不是 null/undefined)

const name = ''
console.log(name || '匿名')  // '匿名'
console.log(name ?? '匿名')  // ''
```

实战: 默认值设置场景

```
// 用户配置
const config = {
  theme: undefined,
  pageSize: 0,
  showSidebar: false
}

// 使用 ?? 保留有意义的值
const theme = config.theme ?? 'light' // 'light'
const pageSize = config.pageSize ?? 20 // 0 (用户设置每页0条, 有意义)
const showSidebar = config.showSidebar ?? true // false (用户关闭了侧边栏)

// 结合可选链
const userName = response?.data?.user?.name ?? '访客'
```

3 Top-level await

ES2022特性

Top-level await 允许在模块的顶层使用 `await` 关键字, 无需包裹在 `async` 函数中。

```
// 以前需要包裹在 async 函数中
(async () => {
  const data = await fetch('/api/data').then(r => r.json())
  console.log(data)
})();

// 现在可以直接在模块顶层使用
// module.mjs
const response = await fetch('/api/data')
const data = await response.json()
export default data
```

在 Module 中的使用

```
// 仅 ES Module 支持, script 需加 type="module"
// <script type="module" src="app.js"></script>

// config.mjs
export const config = await loadConfig()

// app.mjs
import { config } from './config.mjs'
console.log(config) // 等 config 加载完成后再执行
```

动态导入结合

```
// 动态导入 + top-level await
const module = await import(`./locale/${navigator.language}.js`)

// 条件加载
const lodash = await (Condition ? import('lodash') : import('lodash-es'))
```

实战: 模块初始化依赖

```
// db.mjs - 数据库连接初始化
export const db = await createConnection({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT
})

// service.mjs - 依赖 db 模块
import { db } from './db.mjs'
// 此模块会自动等待 db 初始化完成
export const users = await db.query('SELECT * FROM users')
```

4 Promise 新方法

Promise.allSettled()

等待所有 Promise 完成（无论成功或失败），返回每个 Promise 的结果状态。

```
const promises = [
  fetch('/api/a'),
  fetch('/api/b'),
  fetch('/api/c')
]

const results = await Promise.allSettled(promises)
results.forEach(result => {
  if (result.status === 'fulfilled') {
    console.log('成功:', result.value)
  } else {
    console.log('失败:', result.reason)
  }
})
```

Promise.any()

接收一组 Promise，返回**第一个成功（fulfilled）**的结果。如果全部失败，则返回 `AggregateError`。

```
const promises = [
  fetch('/api/slow').then(r => r.json()),
  fetch('/api/fast').then(r => r.json()),
  fetch('/api/cache').then(r => r.json())
]

// 返回最先成功的
Promise.any(promises)
  .then(firstResult => console.log('最快响应:', firstResult))
  .catch(err => console.log('全部失败:', err.errors))
```

Promise.withResolvers()

创建一个 Promise 并同时暴露 resolve 和 reject 方法，避免嵌套。

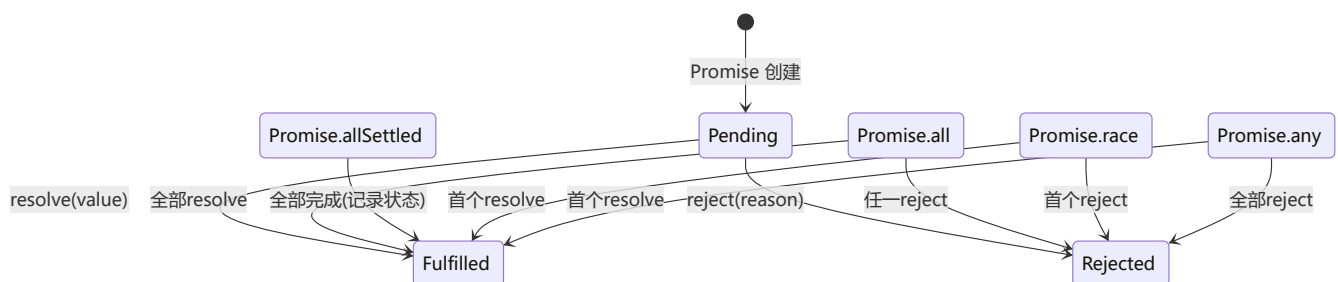
```
// 传统写法
function createPromise() {
  let resolve, reject
  const promise = new Promise((res, rej) => {
    resolve = res
    reject = rej
  })
  return { promise, resolve, reject }
}

// 新写法
const { promise, resolve, reject } = Promise.withResolvers()
```

```
// 使用场景：事件驱动
const { promise: loaded, resolve: onLoaded } = Promise.withResolvers()
img.onload = () => onLoaded(img)
await loaded
```

对比表

方法	行为	短路条件	返回结果
<code>Promise.all</code>	全部完成	任一拒绝	所有值 / 首个拒绝原因
<code>Promise.allSettled</code>	全部完成	不短路	每个 Promise 的 {status, value/reason}
<code>Promise.race</code>	首个完成	首个敲定	首个结果（不论成功/失败）
<code>Promise.any</code>	首个成功	首个成功	首个成功值 / AggregateError



5 structuredClone() 深度克隆

浏览器原生深度克隆

`structuredClone()` 是浏览器原生提供的深度克隆函数，使用结构化克隆算法。

```
const original = {
  name: '张三',
  age: 30,
  hobbies: ['reading', 'coding'],
  address: { city: '北京', district: '海淀' },
  birth: new Date('1994-01-01'),
  regex: /hello/gi,
  map: new Map([['key1', 'value1']]),
  set: new Set([1, 2, 3])
}

const cloned = structuredClone(original)
console.log(cloned === original) // false
console.log(cloned.hobbies === original.hobbies) // false
console.log(cloned.address === original.address) // false
console.log(cloned.birth === original.birth) // false
```

与JSON.parse(JSON.stringify()) 的对比

```
const obj = {
  date: new Date('2024-01-01'),
  regex: /test/gi,
  func: () => {},
  undef: undefined,
  nan: NaN,
  infinity: Infinity,
  map: new Map([[ 'a', 1 ]])
}

// JSON 方式
const jsonClone = JSON.parse(JSON.stringify(obj))
console.log(jsonClone.date)      // "2024-01-01T00:00:00.000Z" (字符串)
console.log(jsonClone.regex)     // {} (空对象)
console.log(jsonClone.func)      // 丢失
console.log(jsonClone.undef)     // 丢失
console.log(jsonClone.map)       // {} (空对象)

// structuredClone 方式
const structClone = structuredClone(obj)
console.log(structClone.date)    // Date 对象 (保留类型)
console.log(structClone.regex)   // RegExp 对象
console.log(structClone.map)     // Map 对象
console.log(structClone.nan)     // NaN
console.log(structClone.infinity) // Infinity
```

支持的数据类型

数据类型	JSON.parse	structuredClone
Object/Array	✓	✓
Date	✗ 转为字符串	✓ 保留 Date 对象
RegExp	✗ 转为空对象	✓
Map	✗ 转为空对象	✓
Set	✗ 转为空对象	✓
Blob/File	✗	✓
TypedArray	✗	✓
Function	✗ 丢失	✗ 抛出异常
Symbol	✗ 丢失	✗ 抛出异常
DOM 元素	✗	✗ 抛出异常

限制条件

```
// ✖ 不支持 Function
structuredClone({ fn: () => {} }) // DOMException

// ✖ 不支持 Symbol
structuredClone({ sym: Symbol('a') }) // DOMException

// ✖ 不支持 DOM 节点
structuredClone(document.body) // DOMException

// ✖ 不支持 Error
structuredClone(new Error('msg')) // DOMException

// ✖ 循环引用
const cyclic = {}
cyclic.self = cyclic
structuredClone(cyclic) // DOMException
```

6 新的 Set 方法

Set.prototype.intersection() - 交集

```
const set1 = new Set([1, 2, 3, 4])
const set2 = new Set([3, 4, 5, 6])

const intersection = set1.intersection(set2)
console.log([...intersection]) // [3, 4]
```

Set.prototype.union() - 并集

```
const set1 = new Set([1, 2, 3])
const set2 = new Set([3, 4, 5])

const union = set1.union(set2)
console.log([...union]) // [1, 2, 3, 4, 5]
```

Set.prototype.difference() - 差集

```
const set1 = new Set([1, 2, 3, 4])
const set2 = new Set([3, 4, 5, 6])

const difference = set1.difference(set2)
console.log([...difference]) // [1, 2]
```

Set.prototype.symmetricDifference() - 对称差集

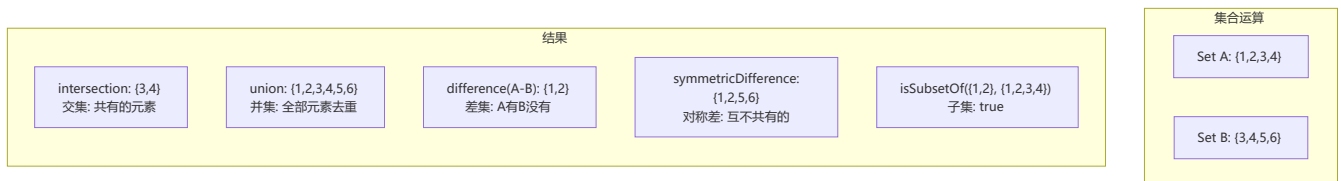
```
const set1 = new Set([1, 2, 3, 4])
const set2 = new Set([3, 4, 5, 6])

const symDiff = set1.symmetricDifference(set2)
console.log([...symDiff]) // [1, 2, 5, 6]
```

Set.prototype.isSubsetOf() - 子集判断

```
const set1 = new Set([1, 2])
const set2 = new Set([1, 2, 3, 4])

console.log(set1.isSubsetOf(set2)) // true
console.log(set2.isSubsetOf(set1)) // false
```



7 Array 和 Object 新方法

Array.fromAsync()

异步版本的 `Array.from()`，用于从异步可迭代对象创建数组。

```
// 异步生成器
async function* asyncGenerator() {
  yield 1
  yield 2
  yield 3
}

const arr = await Array.fromAsync(asyncGenerator())
console.log(arr) // [1, 2, 3]

// 从 Promise 数组
const promises = [Promise.resolve(1), Promise.resolve(2), Promise.resolve(3)]
const result = await Array.fromAsync(promises)
console.log(result) // [1, 2, 3]
```

Iterator Helper

为迭代器提供链式调用的 `map`、`filter`、`take`、`drop`、`flatMap`、`reduce` 方法。

```
function* naturals() {
  let i = 1
  while (true) yield i++
}
```

```

}

const result = naturals()
  .map(x => x * 2)           // 每个元素乘以2
  .filter(x => x > 5)        // 过滤大于5
  .take(3)                  // 取前3个
  .drop(1)                  // 跳过第1个
  .toArray()                // 转为数组

console.log(result) // [8, 10]

// flatMap
function* nums() {
  yield 1; yield 2; yield 3
}

const flat = nums()
  .flatMap(x => [x, x * 10])
  .toArray()

console.log(flat) // [1, 10, 2, 20, 3, 30]

// reduce
const sum = naturals()
  .take(5)
  .reduce((acc, x) => acc + x, 0)

console.log(sum) // 15

```

Object.groupBy() / Map.groupBy()

```

const inventory = [
  { name: '苹果', type: '水果', quantity: 10 },
  { name: '香蕉', type: '水果', quantity: 20 },
  { name: '鲫鱼', type: '水产', quantity: 5 },
  { name: '草鱼', type: '水产', quantity: 8 }
]

// 按类型分组 -> Object
const byType = Object.groupBy(inventory, item => item.type)
console.log(byType)
// {
//   水果: [{ name: '苹果', ... }, { name: '香蕉', ... }],
//   水产: [{ name: '鲫鱼', ... }, { name: '草鱼', ... }]
// }

// 按数量范围分组 -> Map
const byQuantityRange = Map.groupBy(inventory, item => {
  return item.quantity > 10 ? '充足' : '不足'
})
console.log(byQuantityRange)
// Map { '不足' => [{苹果}, {鲫鱼}, {草鱼}], '充足' => [{香蕉}] }

```


Array.prototype.findLast() / findLastIndex()

从数组末尾开始查找。

```
const arr = [1, 2, 3, 4, 5, 3, 2, 1]

// 查找最后一个大于2的元素
const lastLarge = arr.findLast(x => x > 2)
console.log(lastLarge) // 3 (不是5, 因为是3是最后一个 >2 的)

// 查找最后一个大于2的元素索引
const lastLargeIndex = arr.findLastIndex(x => x > 2)
console.log(lastLargeIndex) // 5

// 对比
console.log(arr.find(x => x > 2)) // 3 (正向第一个)
console.log(arr.findIndex(x => x > 2)) // 2 (正向第一个索引)
console.log(arr.findLast(x => x > 2)) // 3 (反向第一个)
console.log(arr.findLastIndex(x => x > 2)) // 5 (反向第一个索引)
```

8 Error Cause

Error 构造函数添加 cause 选项

ES2022 引入 `cause` 属性, 用于构建错误链。

```
// 基本用法
const error = new Error('用户不存在', {
  cause: { code: 404, service: 'user-service' }
})

console.log(error.message) // '用户不存在'
console.log(error.cause) // { code: 404, service: 'user-service' }
```

错误链追踪

```
async function getUser(userId) {
  try {
    const response = await fetch(`/api/users/${userId}`)
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`, {
        cause: { status: response.status, url: response.url }
      })
    }
    return await response.json()
  } catch (err) {
    throw new Error('获取用户信息失败', { cause: err })
  }
}

async function getProfile() {
```

```

try {
  const user = await getUser(123)
  return user.profile
} catch (err) {
  // 完整错误链
  console.log(err.message)      // '获取用户信息失败'
  console.log(err.cause.message) // 'HTTP 404'
  console.log(err.cause.cause)   // { status: 404, url: '/api/users/123' }
  throw new Error('页面加载失败', { cause: err })
}
}

```

实战: 多层调用的错误传递

```

// 数据库层
async function queryDatabase(sql) {
  try {
    return await db.query(sql)
  } catch (err) {
    throw new Error('数据库查询失败', {
      cause: { original: err.message, sql, time: new Date() }
    })
  }
}

// 服务层
async function getUserOrders(userId) {
  try {
    return await queryDatabase(`SELECT * FROM orders WHERE user_id = ${userId}`)
  } catch (err) {
    throw new Error('获取用户订单失败', { cause: err })
  }
}

// API 层
async function handler(req, res) {
  try {
    const orders = await getUserOrders(req.params.userId)
    res.json(orders)
  } catch (err) {
    // 最终统一处理, 保留完整错误链
    console.error(err)
    console.error('原因链:', err.cause?.cause?.original)
    res.status(500).json({ error: err.message })
  }
}

```

9 WeakRef 和 FinalizationRegistry

WeakRef 弱引用

`WeakRef` 允许创建对象的弱引用，不阻止垃圾回收。

```
let target = { data: '重要数据' }
const ref = new WeakRef(target)

// 获取弱引用的目标对象
function getData() {
  const obj = ref.deref()
  if (obj) {
    return obj.data
  }
  return '对象已被回收'
}

console.log(getData()) // '重要数据'

// 解除强引用
target = null
// 此时目标可能已被 GC 回收
```

FinalizationRegistry 注册清理回调

```
const registry = new FinalizationRegistry((heldValue) => {
  console.log(`${heldValue} 已被回收`)
})

let obj = { name: '缓存对象' }
registry.register(obj, 'obj的清理标记')

// 释放引用
obj = null
// GC 后输出: 'obj的清理标记 已被回收'
```

使用场景: 内存敏感缓存

```
class WeakCache {
  constructor() {
    this.cache = new Map()
    this.registry = new FinalizationRegistry((key) => {
      // 值被回收后自动清理键
      this.cache.delete(key)
      console.log(`缓存 [${key}] 已清理`)
    })
  }

  set(key, value) {
    this.cache.set(key, new WeakRef(value))
  }
}
```

```

    this.registry.register(value, key)
  }

  get(key) {
    const ref = this.cache.get(key)
    if (ref) {
      const value = ref.deref()
      if (value) return value
      this.cache.delete(key) // 已被回收，清理条目
    }
    return undefined
  }
}

const cache = new WeakCache()
let data = { heavy: new Array(1000000).fill('*') }
cache.set('large', data)
console.log(cache.get('large')) // { heavy: [...] }
data = null // 解除引用，GC 后缓存自动清理

```

注意事项

```

// ⚠ 不要过度依赖 WeakRef
// 1. deref() 的结果不可预测，可能在下一刻就被回收
const ref = new WeakRef(obj)
const obj2 = ref.deref()
// 此时 obj2 可能已经为 undefined

// 2. GC 时机不确定，不同引擎/版本行为不同
// 3. FinalizationRegistry 的回调可能不按注册顺序执行
// 4. 尽量使用 WeakMap/WeakSet 替代 WeakRef

```

10 globalThis

跨环境全局对象

`globalThis` 是 ES2020 引入的标准全局对象，在任何 JavaScript 环境中都能访问。

```

// 在任何环境中都能获得正确的全局对象
console.log(globalThis)

// 全局变量声明
globalThis.appName = 'MyApp'
console.log(appName) // 'MyApp'

```

在浏览器/Node/WebWorker 中获取全局对象

```

// 不同环境下的全局对象
// 浏览器: window
// Node.js: global
// web worker: self

```

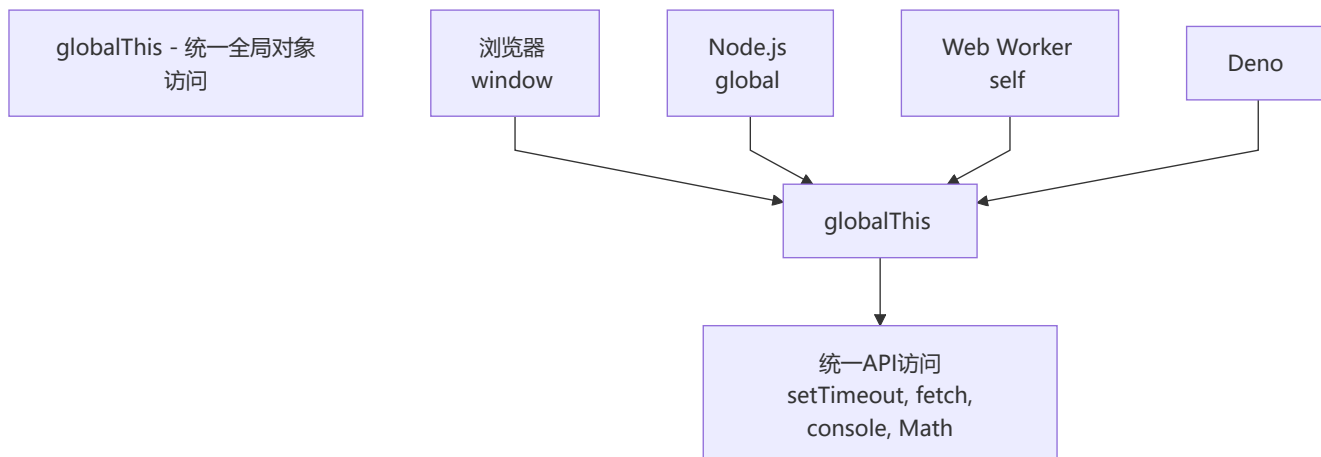
```
// 以前需要判断环境
function getGlobal() {
  if (typeof self !== 'undefined') return self
  if (typeof window !== 'undefined') return window
  if (typeof global !== 'undefined') return global
  throw new Error('无法获取全局对象')
}

// 现在
const env = globalThis
console.log(env === window) // 浏览器: true
console.log(env === global) // Node: true
console.log(env === self) // webworker: true
```

与 window/global/self 的对比

环境	window	global	self	globalThis
浏览器	✓	✗	✓	✓
Node.js	✗	✓	✗	✓
WebWorker	✗	✗	✓	✓
Deno	✗	✗	✗	✓

```
// polyfill (仅在不支持的环境中需要)
// 实际上现代浏览器/Node都已支持
if (!globalThis) {
  Object.defineProperty(Object.prototype, 'globalThis', {
    get() {
      // 通过 Function 构造函数获取全局对象
      return Function('return this')()
    },
    configurable: true
  })
}
```



十二、浏览器 Web API

1 IntersectionObserver

概念和作用

`IntersectionObserver` 提供了一种异步观察目标元素与祖先元素或视口（viewport）交叉状态的方法。用于检测元素是否可见。

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      console.log('元素进入视口:', entry.target)
    }
  })
})

const target = document.querySelector('.lazy-image')
observer.observe(target)

// 停止观察
// observer.unobserve(target)
// observer.disconnect()
```

配置项

```
const observer = new IntersectionObserver(callback, {
  // 目标与视口交叉比例，0~1 或数组 [0, 0.5, 1]
  threshold: 0.5,

  // 视口偏移量，类似 CSS margin
  rootMargin: '10px 20px 30px 40px',

  // 指定父级元素作为视口（默认使用浏览器视口）
  root: document.querySelector('.scroll-container'),

  // 延迟触发时机
  delay: 100,

  // 是否跟踪实际可见性（实验性）
  trackVisibility: false
})
```

实战: 图片懒加载

```
const lazyImages = document.querySelectorAll('img[data-src]')

const imageObserver = new IntersectionObserver((entries, observer) => {
  entries.forEach(entry => {
    if (!entry.isIntersecting) return
```

```

    const img = entry.target
    img.src = img.dataset.src
    img.onload = () => {
      img.classList.add('loaded')
    }
    img.removeAttribute('data-src')
    observer.unobserve(img)
  })
}, {
  rootMargin: '100px', // 提前100px加载
  threshold: 0.01
})

lazyImages.forEach(img => imageObserver.observe(img))

```

实战: 无限滚动

```

const sentinel = document.querySelector('#scroll-sentinel')

const scrollObserver = new IntersectionObserver(async (entries) => {
  const entry = entries[0]
  if (entry.isIntersecting && !isLoading) {
    isLoading = true
    try {
      const newData = await fetchMoreData()
      appendData(newData)
    } finally {
      isLoading = false
    }
  }
}, {
  rootMargin: '200px'
})

scrollObserver.observe(sentinel)

```

实战: 曝光埋点

```

class ExposureTracker {
  constructor(options = {}) {
    this.reported = new WeakSet()
    this.observer = new IntersectionObserver((entries) => {
      entries.forEach(entry => {
        if (entry.isIntersecting && !this.reported.has(entry.target)) {
          this.reported.add(entry.target)
          const eventData = entry.target.dataset.track
          this.report(eventData)
        }
      })
    })
  }, {
    threshold: options.threshold ?? 0.5
  }
}

```

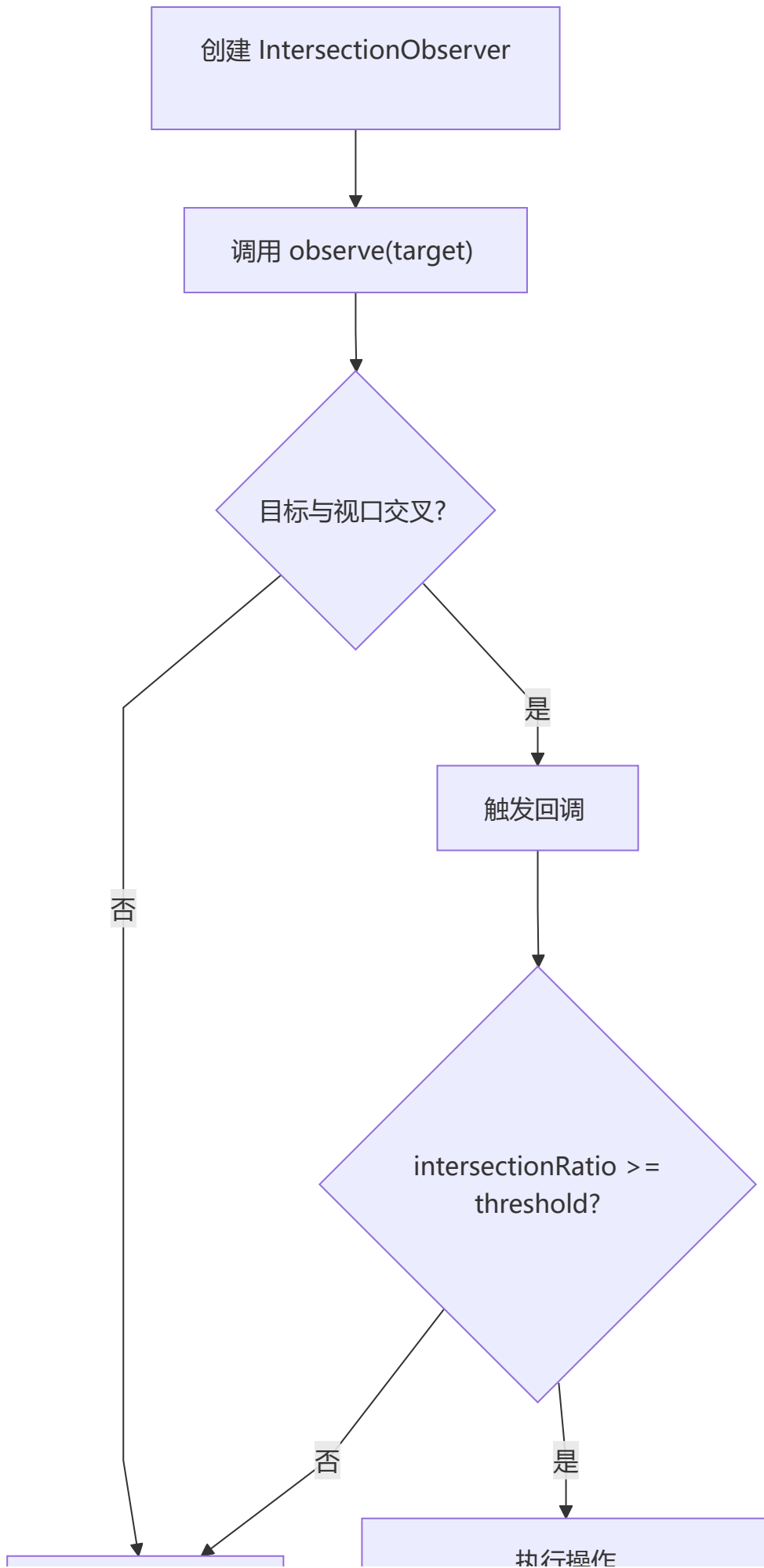
```
    })
  }

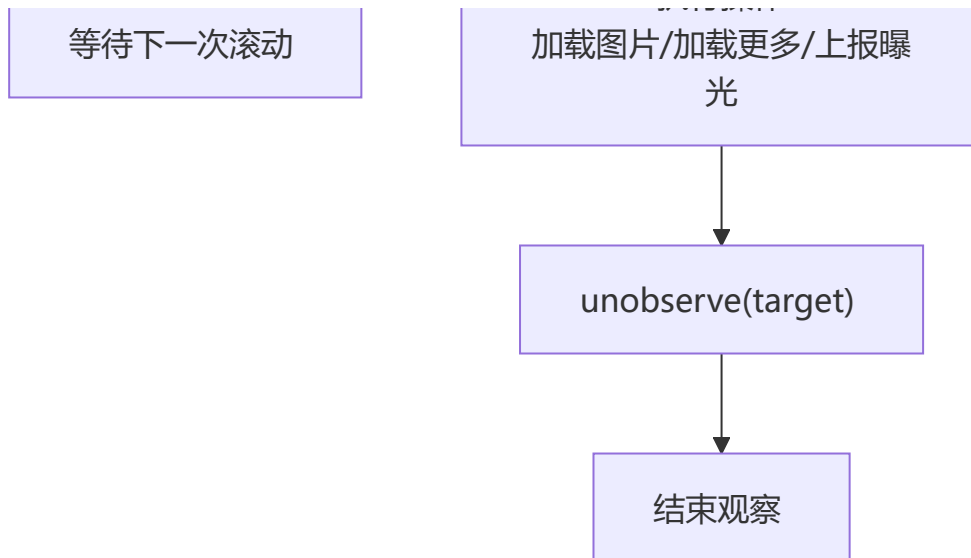
  observe(element) {
    this.observer.observe(element)
  }

  report(data) {
    // 发送埋点请求
    navigator.sendBeacon('/api/track', JSON.stringify({
      element: data,
      timestamp: Date.now()
    }))
  }

  destroy() {
    this.observer.disconnect()
  }
}

// 使用
const tracker = new ExposureTracker({ threshold: 0.3 })
document.querySelectorAll('[data-track]').forEach(e1 => tracker.observe(e1))
```



2 ResizeObserver

监听元素尺寸变化

`ResizeObserver` 用于监听元素的尺寸变化，提供元素内容盒（content-box）、边框盒（border-box）的精确尺寸。

```
const observer = new ResizeObserver((entries) => {
  entries.forEach(entry => {
    const { width, height } = entry.contentBoxSize[0]
    console.log(`${entry.target.id}: ${width} x ${height}`)
  })
})

const element = document.querySelector('.resizable')
observer.observe(element)

// 观察边框盒
observer.observe(element, { box: 'border-box' })
```

与 window.resize 的区别

```
// window.resize 只能监听窗口变化
window.addEventListener('resize', () => {
  console.log('窗口大小变化')
})

// ResizeObserver 可以监听任意元素
const resizeObserver = new ResizeObserver((entries) => {
  entries.forEach(entry => {
    // 元素尺寸变化（包括由内部内容变化引起的）
    console.log(`元素 ${entry.target.id} 尺寸变化:`, entry.contentRect)
  })
})

// 适用场景
```

```
const sidebar = document.querySelector('#sidebar')
const chartContainer = document.querySelector('#chart')
const textarea = document.querySelector('#auto-resize')

resizeObserver.observe(sidebar)
resizeObserver.observe(chartContainer)
resizeObserver.observe(textarea)
```

对比项	window.resize	ResizeObserver
监听对象	仅 window	任意元素
触发时机	窗口大小改变	元素内容/边框盒改变
性能	高频触发，可能卡顿	批量处理，性能更好
精确度	仅获取窗口尺寸	获取 contentBox/borderBox/devicePixelContentBox
触发原因	仅用户调整窗口	元素尺寸变化（含 JS 修改、子元素变化等）

实战: 自适应组件

```
class ResponsiveComponent {
  constructor(element) {
    this.element = element
    this.observer = new ResizeObserver(this.handleResize.bind(this))
    this.observer.observe(element)
  }

  handleResize(entries) {
    const entry = entries[0]
    const { width, height } = entry.contentRect

    // 根据宽度切换布局
    if (width > 800) {
      this.element.classList.add('layout-wide')
      this.element.classList.remove('layout-narrow')
    } else if (width > 400) {
      this.element.classList.add('layout-medium')
      this.element.classList.remove('layout-wide', 'layout-narrow')
    } else {
      this.element.classList.add('layout-narrow')
      this.element.classList.remove('layout-wide', 'layout-medium')
    }

    // 更新图表或画布
    if (this.chart) {
      this.chart.resize({ width, height })
    }
  }

  destroy() {
    this.observer.disconnect()
  }
}
```

```
}  
}  
  
// 使用  
const chartwidget = new ResponsiveComponent(document.querySelector('#chart-widget'))
```

3 MutationObserver

监听 DOM 变化

`MutationObserver` 用于监听 DOM 树的变更事件，包括节点增删、属性变化、文本内容变化等。

```
const observer = new MutationObserver((mutations) => {  
  mutations.forEach(mutation => {  
    console.log('变更类型:', mutation.type)  
    console.log('目标节点:', mutation.target)  
  
    if (mutation.type === 'childList') {  
      console.log('新增节点:', mutation.addedNodes)  
      console.log('移除节点:', mutation.removeNodes)  
    }  
  
    if (mutation.type === 'attributes') {  
      console.log('属性名:', mutation.attributeName)  
      console.log('旧值:', mutation.oldValue)  
    }  
  
    if (mutation.type === 'characterData') {  
      console.log('文本变化:', mutation.oldValue, '→', mutation.target.textContent)  
    }  
  })  
})
```

配置项

```
const targetNode = document.querySelector('#observable-area')  
  
observer.observe(targetNode, {  
  // 监听子节点增删  
  childList: true,  
  
  // 监听属性变化  
  attributes: true,  
  
  // 监听文本变化  
  characterData: true,  
  
  // 是否监听所有后代节点  
  subtree: true,  
  
  // 是否记录属性旧值  
  attributeOldValue: true,
```

```

// 是否记录文本旧值
characterDataOldValue: true,

// 监听的特定属性列表（默认监听所有）
attributeFilter: ['class', 'style', 'data-*']
})

```

实战: 监听子元素变化

```

// 动态列表监控
class DynamicListMonitor {
  constructor(listElement, options = {}) {
    this.list = listElement
    this.onItemAdd = options.onItemAdd || (() => {})
    this.onItemRemove = options.onItemRemove || (() => {})

    this.observer = new MutationObserver((mutations) => {
      mutations.forEach(mutation => {
        mutation.addedNodes.forEach(node => {
          if (node.nodeType === 1) { // 元素节点
            this.onItemAdd(node)
          }
        })
        mutation.removedNodes.forEach(node => {
          if (node.nodeType === 1) {
            this.onItemRemove(node)
          }
        })
      })
    })
  }

  this.observer.observe(listElement, {
    childList: true,
    subtree: false
  })
}

destroy() {
  this.observer.disconnect()
}
}

// 使用: 监控聊天消息列表
const chatMonitor = new DynamicListMonitor(
  document.querySelector('#chat-messages'),
  {
    onItemAdd: (node) => {
      console.log('新消息:', node.textContent)
      node.classList.add('message-enter')
      // 自动滚动到底部
      node.parentElement.scrollTop = node.parentElement.scrollHeight
    }
  }
)

```

```
    },
    onItemRemove: (node) => {
      console.log('消息被移除:', node.id)
    }
  }
}
```

4 AbortController

中止请求

`AbortController` 提供一个 `signal` 属性和 `abort()` 方法，用于中止正在进行 DOM 操作或网络请求。

```
const controller = new AbortController()
const signal = controller.signal

// 监听中止事件
signal.addEventListener('abort', () => {
  console.log('请求已中止:', signal.aborted ? '是' : '否')
  console.log('中止原因:', signal.reason)
})

// 发起可中止的请求
fetch('/api/large-data', { signal })
  .then(response => response.json())
  .catch(err => {
    if (err.name === 'AbortError') {
      console.log('请求被用户取消')
    } else {
      console.error('其他错误:', err)
    }
  })

// 在超时或其他条件下中止请求
setTimeout(() => controller.abort('用户取消了请求'), 5000)
```

与 fetch 配合

```
function fetchWithTimeout(url, timeout = 5000) {
  const controller = new AbortController()
  const timeoutId = setTimeout(() => controller.abort('请求超时'), timeout)

  return fetch(url, { signal: controller.signal })
    .finally(() => clearTimeout(timeoutId))
}

// 使用
try {
  const response = await fetchWithTimeout('https://api.example.com/data', 3000)
  const data = await response.json()
  console.log(data)
} catch (err) { }
```

```
if (err.name === 'AbortError') {
  console.log('请求超时或被取消')
}
}
```

实战: 取消重复请求

```
class RequestManager {
  constructor() {
    this.pendingRequests = new Map()
  }

  async request(url, options = {}) {
    // 如果已有相同 url 的请求, 取消旧的
    if (this.pendingRequests.has(url)) {
      const oldController = this.pendingRequests.get(url)
      oldController.abort('请求被新的请求替代')
      this.pendingRequests.delete(url)
    }

    const controller = new AbortController()
    this.pendingRequests.set(url, controller)

    try {
      const response = await fetch(url, {
        ...options,
        signal: controller.signal
      })
      return await response.json()
    } finally {
      this.pendingRequests.delete(url)
    }
  }

  cancelAll(reason = '页面离开') {
    this.pendingRequests.forEach(controller => {
      controller.abort(reason)
    })
    this.pendingRequests.clear()
  }
}

// 搜索防抖 + 取消旧请求
const requestManager = new RequestManager()
const searchInput = document.querySelector('#search')

let debounceTimer
searchInput.addEventListener('input', (e) => {
  clearTimeout(debounceTimer)
  debounceTimer = setTimeout(() => {
    requestManager.request(`/api/search?q=${e.target.value}`)
  }, 300)
```

```
}))
```

5 PerformanceObserver

性能指标监控

`PerformanceObserver` 用于监测性能度量事件，获取页面性能数据。

```
// 基本用法
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach(entry => {
    console.log(`${entry.name}: ${entry.duration}ms`)
  })
})

// 观察特定类型的性能条目
observer.observe({ entryTypes: ['resource', 'navigation', 'paint', 'largest-contentful-paint'] })
```

LCP/FID/CLS 监控

```
// LCP - Largest Contentful Paint (最大内容绘制)
const lcpObserver = new PerformanceObserver((list) => {
  const entries = list.getEntries()
  const lastEntry = entries[entries.length - 1]
  console.log('LCP:', lastEntry.startTime, 'ms')
  console.log('LCP元素:', lastEntry.element)
})
lcpObserver.observe({ type: 'largest-contentful-paint', buffered: true })

// FID - First Input Delay (首次输入延迟)
const fidObserver = new PerformanceObserver((list) => {
  list.getEntries().forEach(entry => {
    console.log('FID:', entry.processingStart - entry.startTime, 'ms')
    console.log('交互类型:', entry.name)
  })
})
fidObserver.observe({ type: 'first-input', buffered: true })

// CLS - Cumulative Layout Shift (累计布局偏移)
let clsValue = 0
let clsEntries = []
const clsObserver = new PerformanceObserver((list) => {
  list.getEntries().forEach(entry => {
    if (!entry.hadRecentInput) {
      clsValue += entry.value
      clsEntries.push(entry)
      console.log('CLS值:', clsValue, '来源:', entry.sources)
    }
  })
})
```



```
clsobserver.observe({ type: 'layout-shift', buffered: true })
```

实战: 性能埋点

```
class PerformanceMonitor {
  constructor(reportUrl) {
    this.reportUrl = reportUrl
    this.metrics = {}
    this.initObservers()
  }

  initObservers() {
    // 页面导航性能
    const navObserver = new PerformanceObserver((list) => {
      const [entry] = list.getEntries()
      this.metrics.navigation = {
        dns: entry.domainLookupEnd - entry.domainLookupStart,
        tcp: entry.connectEnd - entry.connectStart,
        ttfb: entry.responseStart - entry.requestStart,
        domReady: entry.domContentLoadedEventEnd - entry.domContentLoadedEventStart,
        loadTime: entry.loadEventEnd - entry.loadEventStart
      }
    })
    navObserver.observe({ type: 'navigation', buffered: true })

    // 资源加载
    const resObserver = new PerformanceObserver((list) => {
      list.getEntries().forEach(entry => {
        if (entry.initiatorType === 'script' || entry.initiatorType === 'link' ||
            entry.initiatorType === 'img') {
          this.metrics.resources ||= {}
          this.metrics.resources[entry.name] = entry.duration
        }
      })
    })
    resObserver.observe({ type: 'resource', buffered: true })

    // web vitals
    const vitalsObserver = new PerformanceObserver((list) => {
      list.getEntries().forEach(entry => {
        if (entry.entryType === 'largest-contentful-paint') {
          this.metrics.LCP = entry.startTime
        }
        if (entry.entryType === 'first-input') {
          this.metrics.FID = entry.processingStart - entry.startTime
        }
        if (entry.entryType === 'layout-shift' && !entry.hadRecentInput) {
          this.metrics.CLS = (this.metrics.CLS || 0) + entry.value
        }
      })
    })
  }
}
```

```

    vitalsoobserver.observe({ entryTypes: ['largest-contentful-paint', 'first-input',
'layout-shift'], buffered: true })
  }

  report() {
    // 页面关闭前上报
    if (navigator.sendBeacon) {
      navigator.sendBeacon(this.reportUrl, JSON.stringify(this.metrics))
    }
  }
}

// 使用
const monitor = new PerformanceMonitor('/api/perf-report')
document.addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'hidden') {
    monitor.report()
  }
})

```

6 BroadcastChannel

跨标签页通信

`BroadcastChannel` 允许同源的不同浏览上下文（标签页、iframe、Worker）之间进行消息通信。

```

// 页面 A - 发送消息
const channelA = new BroadcastChannel('app_channel')
channelA.postMessage({ type: 'USER_LOGIN', data: { userId: 123, name: '张三' } })

// 页面 B - 接收消息
const channelB = new BroadcastChannel('app_channel')
channelB.onmessage = (event) => {
  console.log('收到消息:', event.data)
  // { type: 'USER_LOGIN', data: { userId: 123, name: '张三' } }
}

// 清理
// channelA.close()
// channelB.close()

```

与 postMessage / localStorage 对比

```

// 方案1: BroadcastChannel
const bc = new BroadcastChannel('channel')
bc.postMessage('hello')
bc.onmessage = (e) => console.log(e.data)
// 优点: 专门用于同源跨标签页通信, API 简洁
// 缺点: 仅支持同源

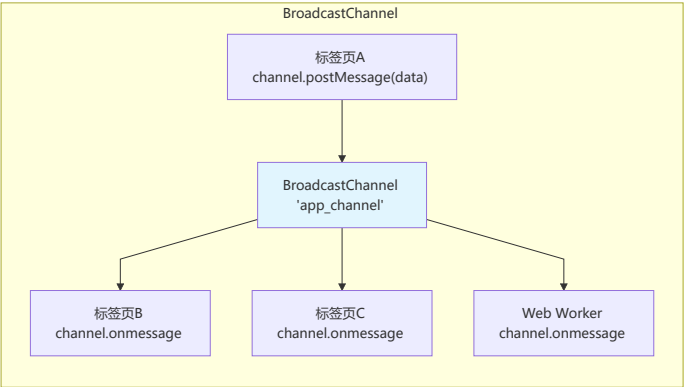
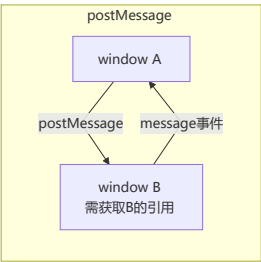
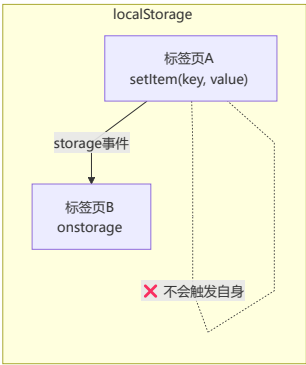
// 方案2: window.postMessage
// window.opener.postMessage('hello', '*')

```

```
// window.addEventListener('message', (e) => console.log(e.data))
// 优点：支持跨域
// 缺点：需要获取窗口引用，安全性需注意 origin

// 方案3: localStorage
window.addEventListener('storage', (e) => {
  if (e.key === 'shared-data') {
    console.log('数据变化:', e.newValue)
  }
})
localStorage.setItem('shared-data', JSON.stringify({ msg: 'hello' }))
// 优点：简单易用，不需要建立连接
// 缺点：存储大小限制，存储事件不会在触发页面触发，会序列化
```

对比项	BroadcastChannel	postMessage	localStorage
通信范围	同源标签页/iframe/Worker	任意窗口（可跨域）	同源标签页
API 复杂度	简单	中等	简单
数据格式	结构化克隆	结构化克隆	仅字符串
是否需要引用	不需要	需要窗口对象	不需要
事件接收	所有同源页面	指定窗口	除自身外的同源页面
安全性	同源限制	需验证 origin	同源限制



```
// 实战：跨标签页状态同步
class CrossTabState {
  constructor(channelName = 'app_state') {
    this.channel = new BroadcastChannel(channelName)
    this.listeners = new Map()

    this.channel.onmessage = (event) => {
      const { type, payload } = event.data
      const callbacks = this.listeners.get(type) || []
      callbacks.forEach(cb => cb(payload, event.origin))
    }
  }
}
```

```
// 发送消息到所有同源页面
broadcast(type, payload) {
  this.channel.postMessage({ type, payload })
}

// 监听特定消息类型
on(type, callback) {
  if (!this.listeners.has(type)) {
    this.listeners.set(type, [])
  }
  this.listeners.get(type).push(callback)
}

// 取消监听
off(type, callback) {
  const callbacks = this.listeners.get(type)
  if (callbacks) {
    this.listeners.set(type, callbacks.filter(cb => cb !== callback))
  }
}

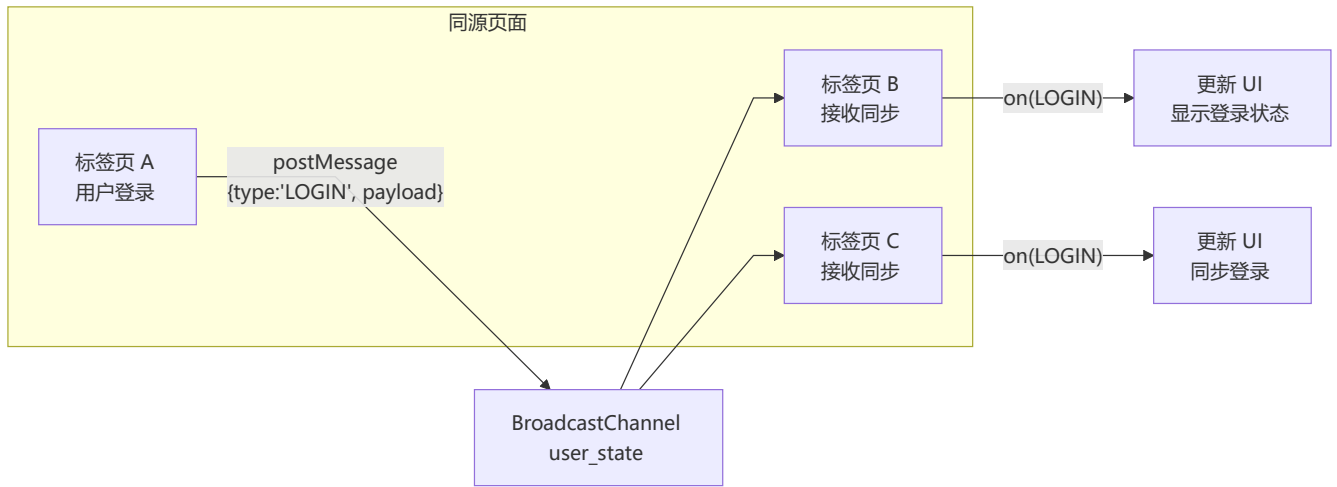
close() {
  this.channel.close()
  this.listeners.clear()
}
}

// 使用：用户登录状态同步
const stateSync = new CrossTabState('user_state')

// 页面 A：用户登录
stateSync.broadcast('LOGIN', { userId: 1, name: '张三', token: 'xxx' })

// 页面 B：监听登录事件
stateSync.on('LOGIN', (userData) => {
  console.log(`用户 ${userData.name} 在其他标签页登录了`)
  updateUI(userData)
})

// 页面 C：监听登出事件
stateSync.on('LOGOUT', () => {
  console.log('用户已从其他标签页登出')
  redirectToLogin()
})
```



总结

